第三章 数据据链路层基础

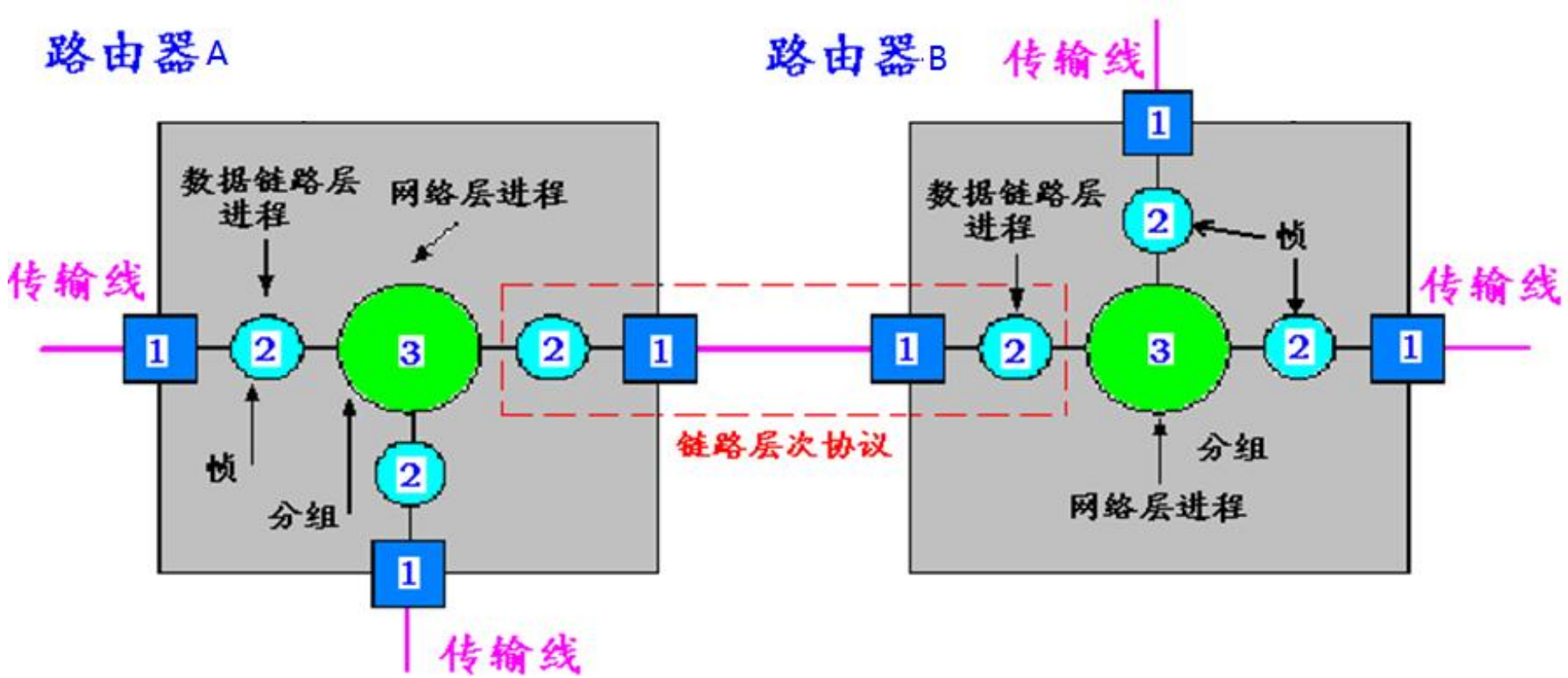
授课教师：邢焕来

主要内容来自：计算机网络教案社区，版权归原作者所有，有增删改

本章目标

- 了解数据链路层在网络体系结构中的位置及基本功能和服务
- 掌握差错检测和纠正的基本原理和典型的编码方法（核心内容）
- 掌握无错信道和有错信道上停等协议的设计和实现方法（核心内容）
- 理解停等协议的性能问题及滑动窗口协议的基本思想
- 掌握回退N和选择重传两种典型滑动窗口协议的工作机制（核心内容）
- 了解点到点链路层协议PPP与PPPoE

OSI数据链路层的作用：点到点的可靠传输



数据链路层的功能：

OSI数据链路层协议，**在不可靠的物理链路上实现可靠的传输**(无差错、无丢失、无重复、按顺序)：

- ✓ **数据成帧**：传输单位：帧；帧同步
- ✓ **差错控制**：纠错；检错
- ✓ **流量控制**：收、发方速率适配
- ✓ **透明传输**：无论上层什么数据，链路层都可同样传输
- ✓ **链路控制**：收发方确认对方是否准备好；数据链路的建立、维持、释放
- ✓ **寻址**：MAC地址

数据链路层的功能：

OSI数据链路层协议，**在不可靠的物理链路上实现可靠的传输**(无差错、无丢失、无重复、按顺序)：

- ✓ **数据成帧**：传输单位：帧；帧同步
- ✓ **差错控制**：纠错；检错
- ✓ **流量控制**：收、发方速率适配
- ✓ **透明传输**：无论上层什么数据，链路层都可同样传输
- ✓ **链路控制**：收发方确认对方是否准备好；数据链路的建立、维持、释放
- ✓ **寻址：**MAC地址

注意，关于“可靠”，这是OSI模型中的要求，实际网络中的数据链路层并不一定保证这一点。
e.g., 802.3 vs 802.11

OSI model

Layer			Protocol data unit (PDU)	Function[20]
Host layers	7	Application	Data	High-level APIs, including resource sharing, remote file access
	6	Presentation		Translation of data between a networking service and an application; including character encoding, data compression and encryption/decryption
	5	Session		Managing communication sessions, i.e., continuous exchange of information in the form of multiple back-and-forth transmissions between two nodes
	4	Transport	Segment, Datagram	Reliable transmission of data segments between points on a network, including segmentation, acknowledgement and multiplexing
Media layers	3	Network	Packet	Structuring and managing a multi-node network, including addressing, routing and traffic control
	2	Data link	Frame	Reliable transmission of data frames between two nodes connected by a physical layer
	1	Physical	Bit, Symbol	Transmission and reception of raw bit streams over a physical medium



本章内容

3.1 数据链路层的设计问题
3.2 差错检测和纠正
3.3 基本的数据链路层协议
3.4 滑动窗口协议
3.5 数据链路协议实例

1. 数据链路层在协议栈中的位置
2. 数据链路层的功能
3. 数据链路层提供的服务
4. 成帧
5. 差错控制
6. 流量控制

数据链路层在协议栈中的位置

- 向下：利用物理层提供的位流服务
- 向上：向网络层提供明确的 (well-defined) 服务接口



参考协议栈

数据链路层的功能

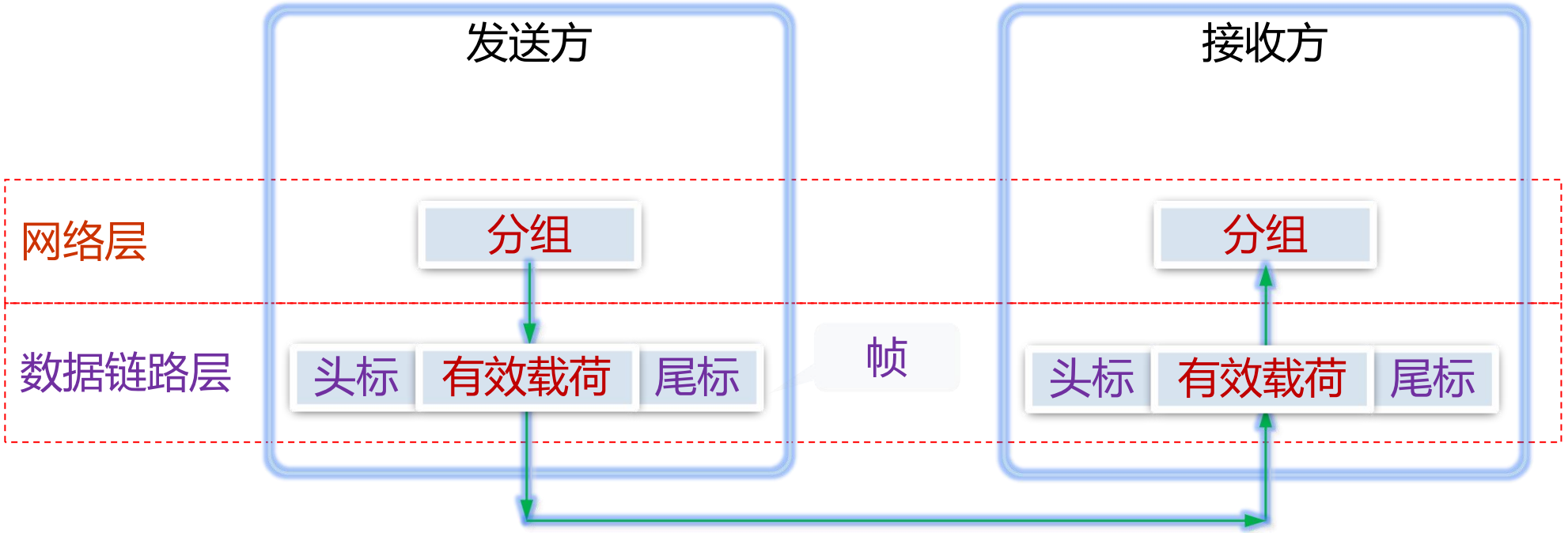
- **成帧 （Framing）**
  - 将比特流划分成“帧”的主要目的是为了检测和纠正物理层在比特传输中可能出现的错误，数据链路层功能需借助“帧”的各个域来实现
- **差错控制 （Error Control）**
  - 处理传输中出现的差错，如位错误、丢失等
- **流量控制 （Flow Control）**
  - 确保发送方的发送速率，不大于接收方的处理速率
    - 避免接收缓冲区溢出

数据链路层提供的服务

- **无确认 无连接 服务（Unacknowledged connectionless）**
  - 接收方不对收到的帧进行确认
  - 适用场景：误码率低的可靠信道；实时通信；
  - 网络实例：以太网
- **有确认 无连接 服务（Acknowledged connectionless）**
  - 每一帧都得到单独的确认
  - 适用场景：不可靠的信道（无线信道）
  - 网络实例：802.11
- **有确认 有连接 服务（Acknowledged connection-oriented）**
  - 适用场景：长延迟的不可靠信道

成帧（Framing）

- 分组 (packet) 与 帧(frame)的关系



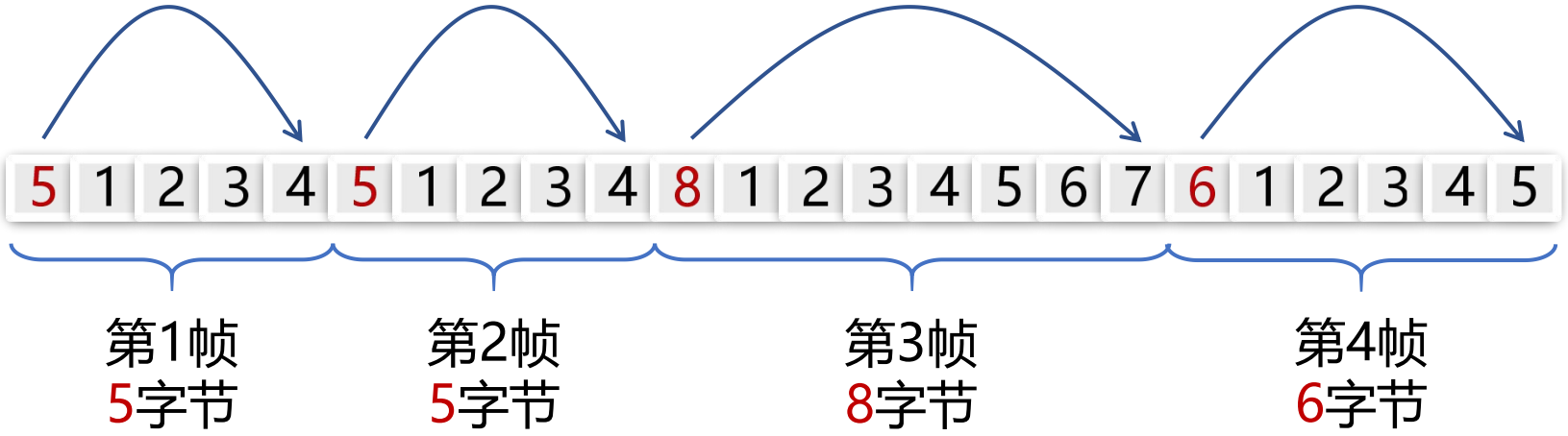
成帧（Framing）

- **帧定界问题**（比特流分成帧）；
- **透明传输问题**（任何字符都可以传输）；

- 关键问题：如何标识一个帧的开始？
  - 接收方必须能从物理层接收的比特流中明确区分出一帧的开始和结束，这个问题被称为帧同步或帧定界
  - 关键：选择何种定界符？定界符出现在数据部分如何处理？
- 成帧（framing）的方式
  - 字节计数法（ Byte count ）
  - 带字节填充的定界符法（ Flag bytes with byte stuffing ）
  - 带比特填充的定界符法（ Flag bits with bit stuffing ）
  - 物理层编码违例（ Physical layer coding violations ）

成帧（Framing）

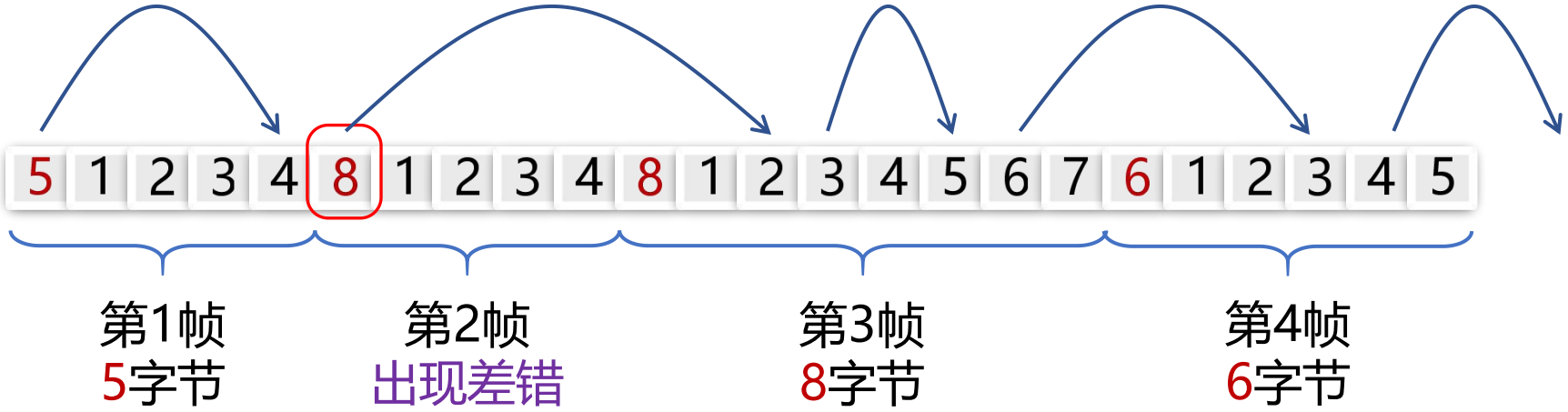
- 字节计数法：无差错传输的情形



问题：如果某个计数字节出错会发生什么情况？

成帧（Framing）

- 字节计数法：出现了一个字节差错的情形



破坏了帧的边界，导致一连串帧的错误

成帧（Framing）

➢ 带字节填充的定界符法

- ASCII字符：帧头DLE STX，帧尾DLE ETX
  - DLE (Data Link Escape);
  - STX (Start of Text);
  - EXT (End of Text)
- 如果字符中出现DLE，就添一个DLE，称为**字符填充**。
- 缺点：完全依赖8位字符，特别是ASCII字符。

联想：
C语言中“\”字符的表示

例： 网络层发出：

A	DLE	B	C

经数据链路层填充：

DLE	STX	A	DLE	DLE	B	C	DLE	ETX

传给接收方网络层：

A	DLE	B	C

成帧（Framing）

- 典型代表是IBM二进制同步通信协议(BSC)。
- 特点是一次传送若干个字符组成的数据块而非一个字符，并规定10个字符作为该数据块的开头与结束标志及整个传输过程的控制信息(通信控制字)。由于被传送的数据块是由字符组成，故被称作面向字符的协议。

- 相关控制字符的ASCII定义：

符号名称	十进制	十六进制
SOH（start of headline 标题开始）	1	01
STX（start of text 正文开始）	2	02
ETX（end of text 正文结束）	3	03
EOT（end of transmission传输结束）	4	04
ENQ（enquiry 请求）	5	05
ACK（acknowledge 收到通知）	6	06
DLE（data link escape 数据链路转义）	16	10
NAK（negative acknowledge 拒绝接收）	21	15
SYN（(synchronous idle 同步空闲）	22	16
ETB（end of trans. block 传输块结束）	23	17

成帧（Framing）

ASCII 字符代码表 一

高四位 / 低四位		ASCII非打印控制字符										ASCII 打印字符												
		0000					0001					0010		0011		0100		0101		0110		0111		
		0					1					2		3		4		5		6		7		
		+进制	字符	ctrl	代码	字符解释	+进制	字符	ctrl	代码	字符解释	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	ctrl
0000	0	0	BLANK NULL	^@	NUL	空	16	►	^P	DLE	数据链路转意	32		48	0	64	@	80	P	96	`	112	p	
0001	1	1	☺	^A	SOH	头标开始	17	◄	^Q	DC1	设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q	
0010	2	2	☻	^B	STX	正文开始	18	↕	^R	DC2	设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r	
0011	3	3	♥	^C	ETX	正文结束	19	!!	^S	DC3	设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s	
0100	4	4	♦	^D	EOT	传输结束	20	¶	^T	DC4	设备控制 4	36	$	52	4	68	D	84	T	100	d	116	t	
0101	5	5	♣	^E	ENQ	查询	21	§	^U	NAK	反确认	37	%	53	5	69	E	85	U	101	e	117	u	
0110	6	6	♠	^F	ACK	确认	22	■	^V	SYN	同步空闲	38	&	54	6	70	F	86	V	102	f	118	v	
0111	7	7	•	^G	BEL	震铃	23	↨	^W	ETB	传输块结束	39	'	55	7	71	G	87	W	103	g	119	w	
1000	8	8	◘	^H	BS	退格	24	↑	^X	CAN	取消	40	(	56	8	72	H	88	X	104	h	120	x	
1001	9	9	○	^I	TAB	水平制表符	25	↓	^Y	EM	媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y	
1010	A	10	◙	^J	LF	换行/新行	26	→	^Z	SUB	替换	42	*	58	:	74	J	90	Z	106	j	122	z	
1011	B	11	♂	^K	VT	竖直制表符	27	←	^[	ESC	转意	43	+	59	;	75	K	91	[	107	k	123	{	
1100	C	12	♀	^L	FF	换页/新页	28	∟	^\	FS	文件分隔符	44	,	60	<	76	L	92	\	108	l	124	|	
1101	D	13	♪	^M	CR	回车	29	↔	^]	GS	组分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	♫	^N	SO	移出	30	▲	^6	RS	记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~	
1111	F	15	☼	^O	SI	移入	31	▼	^-	US	单元分隔符	47	/	63	?	79	O	95	_	111	o	127	Δ	"Back space

注：表中的ASCII字符可以用：ALT + "小键盘上的数字键" 输入

成帧（Framing）

➢ 带**比特**填充的定界符法

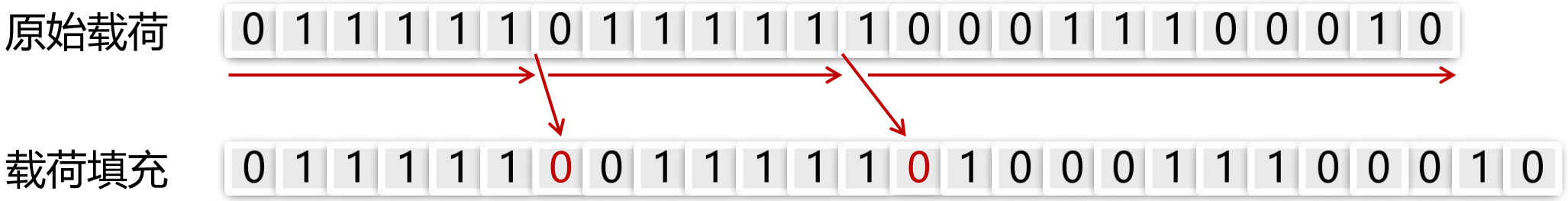
- 定界符：两个0比特之间，连续6个1比特，即01111110，0x7E



问题：如果有效载荷部分包含与“定界符”相同的位组合如何解决?

成帧（Framing）

- 带**比特**填充的定界符法：发送方检查有效载荷
  - 若在有效载荷中出现连续5个1比特，则直接插入1个0比特

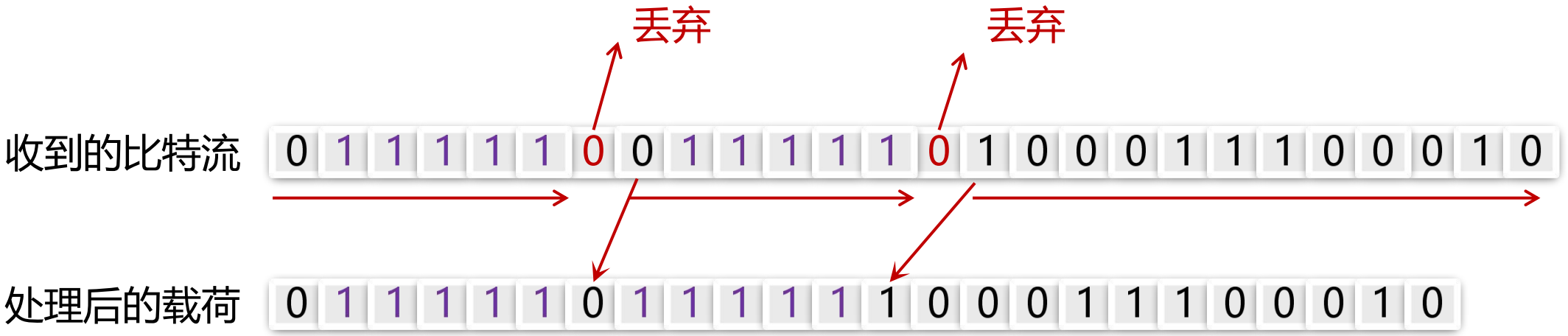


成帧（Framing）

➢ 带**比特**填充的定界符法：接收方的处理

- 若出现连续5个1比特
  - 若下一比特为0，则为有效载荷，直接丢弃0比特
  - 若下一比特为1，则连同后一比特的0，构成定界符，一帧结束

若出现连续5个1比特，则去掉后面的一个“0”



成帧（Framing）

➢ 物理层编码违例：

- **核心思想：选择的定界符不会在数据部分出现**
- 4B/5B编码方案
  - 4比特数据映射成5比特编码，剩余的一半码字（16个码字）未使用，可以用做帧定界符
  - 例如： 00110组合不包含在4B/5B编码中，可做帧定界符
- 前导码
  - 存在很长的 前导码（preamble），可以用作定界符
  - 例如：传统以太网、802.11
- 曼切斯特编码 / 差分曼切斯特编码
  - 正常的信号在周期中间有跳变，持续的高电平（或低电平）为违例码，可以用作定界符
  - 例如：802.5令牌环网

差错控制

- 链路层存在的一个问题：信道的噪声导致数据传输问题
  - 差错（incorrect）：数据发生错误
  - 丢失（lost）：接收方未收到
  - 乱序（out of order）：先发后到，后发先到
  - 重复（repeatedly delivery）：一次发送，多次接收
- 解决方案：差错检测与纠正、确认重传
  - 确认：接收方校验数据（差错校验），并给发送方应答，防止差错
  - 定时器：发送方启动定时器，防止丢失
  - 顺序号：接收方检查序号，防止乱序递交、重复递交

流量控制

- 链路层存在的另一个问题：接收方的处理速率
  - 接收方的接收缓冲区溢出
- 解决方案
  - 基于反馈 (feedback-based) 的流量控制
    - 接收方反馈，发送方调整发送速率
  - 基于速率 (rate-based) 的流量控制
    - 发送方根据内建机制，自行限速

本章内容

3.1 数据链路层的设计问题

3.2 差错检测和纠正

1. 差错检测与纠正概述
2. 典型的纠错码
3. 典型的检错码

3.3 基本的数据链路层协议

3.4 滑动窗口协议

3.5 数据链路协议实例

差错原因

- **噪声干扰：**
  - 随机热噪声：导体电子运动引起，时刻存在，幅度较小，强度与频率无关，频谱很宽。
  - 冲击噪声：引起差错的主要原因，引起相邻数据突发性错。
- **数据处理引起：**
  - 调制、解调
- **信道失真：**
  - 色散：光线透过媒介中间时，因波长关系光学性质改变的一种现象。如光线射入镜片或三棱镜时，因波长的不同产生折射率互异的现象，称之为光的分散或色散。光纤色散主要是因为光源不是单色光引起。
  - 有限带宽：
  - 限幅：
- **其它原因：**
  - 介质连接不良，通信设备数据处理出错……

差错的类型

- 单个错误：分散在各块中
- 突发错误：集中在某个块中
- 例子：块大小： 1000 bits （100个块），出错率是：每个比特 0.001（1‰）
  - 单个错误：分散在各个数据块中
  - 突发错误：集中于一个数据块，整个块都是错误
- 突发错误比单个错误更加难于处理
  - 通常利用处理单个处理的方法来应对突发错误

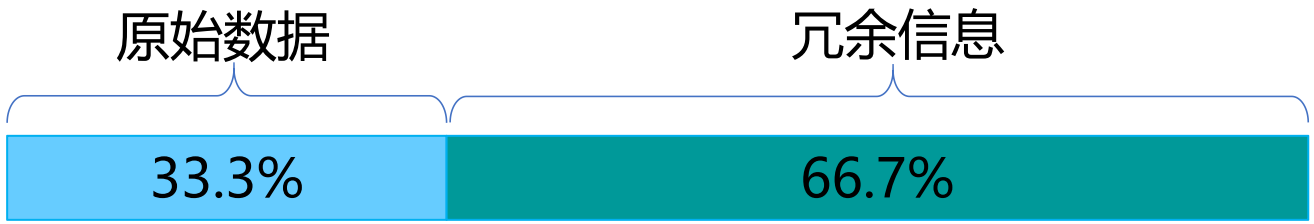
差错检测和纠正概述

- 如何解决信道传输差错问题
  - 通常采用增加冗余信息（或称校验信息）的策略
  - 简单示例：每个比特传三份，如果每比特的三份中有一位出错，可以纠正

0	1	0	0	1	0	1	0	1	0	1	1	0	1
0	1	0	0	1	0	1	0	1	0	1	1	0	1
0	1	0	0	1	0	1	0	1	0	1	1	0	1

携带2/3的冗余信息！

冗余信息量大！



差错检测和纠正概述

- 目标：保证一定差错检测和纠错能力的前提下，如何减少冗余信息量？
- 考虑的问题
  - 信道的特征和传输需求
  - 冗余信息的计算方法、携带的冗余信息量
  - 计算的复杂度等
- 两种主要策略
  - 检错码（error-detecting code）
  - 纠错码（error-correcting code）

差错检测和纠正概述

- 检错码（error-detecting code）
  - 在被发送的数据块中，包含一些冗余信息，但这些信息只能使接收方推断是否发生错误，但不能推断哪位发生错误，接收方可以请求发送方重传数据
  - 主要用在高可靠、误码率较低的信道上，例如光纤链路
  - 偶尔发生的差错，可以通过重传解决差错问题

差错检测和纠正概述

- 纠错码（error-correcting code)
  - 发送方在每个数据块中加入足够的冗余信息，使得接收方能够判断接收到的数据是否有错，并能纠正错误（定位出错的位置）
  - 主要用于错误发生比较频繁的信道上，如无线链路
  - 也经常用于物理层，以及更高层（例如，实时流媒体应用和内容分发)
  - 使用纠错码的技术通常称为前向纠错（FEC，Forward Error Correction)

差错检测和纠正概述

- 码字 (code word)：一个包含m个数据位和r个校验位的n位单元
  - 描述为 (n, m) 码，n=m+r
- 码率 (code rate)：码字中不含冗余部分所占的比例，可以用m/n表示
- 海明距离 (Hamming distance)：两个码字之间不同对应比特的数目
  - 例：0000000000 与0000011111的海明距离为5
  - 如果两个码字的海明距离为d，则需要d个单比特错就可以把一个码字转换成另一个码字
  - 为了检查出d个错（比特错），可以使用海明距离为 **d+1** 的编码
  - 为了纠正d个错，可以使用海明距离为 **2d+1** 的编码

引自：清华大学崔勇老师提供的讲稿

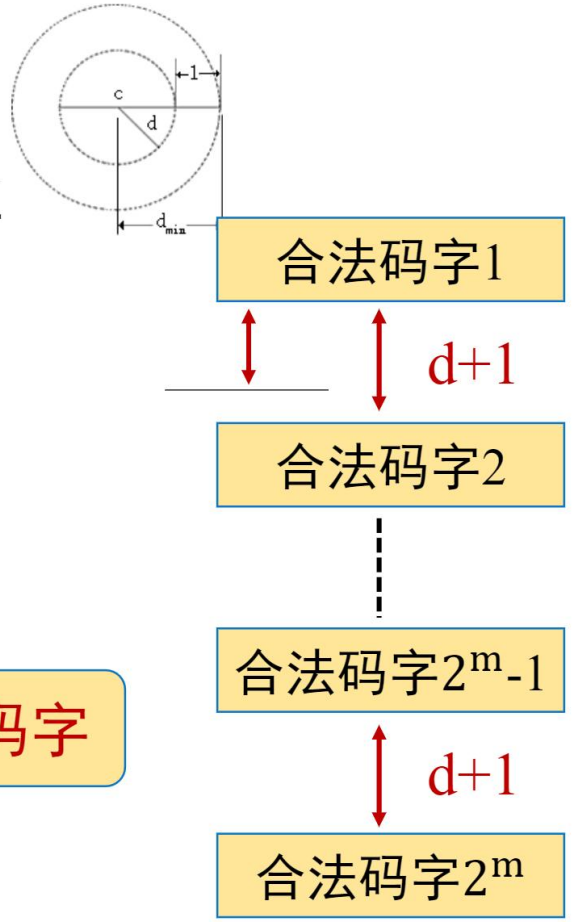
思考题

为了检查出d个错（比特错），可以使用海明距离为 **d+1** 的编码?

思考题

为了检查出d个错（比特错），可以使用海明距离为 **d+1** 的编码?

- 海明距离为d+1的编码能检测出d位差错。
  - 因为在距离为d+1的检验码中，只改变d位的值，不可能产生另一个合法码。如奇偶校验码，海明距离为2，能查出单个错。



系统中既有合法的码字，也有非法（错误）的码字

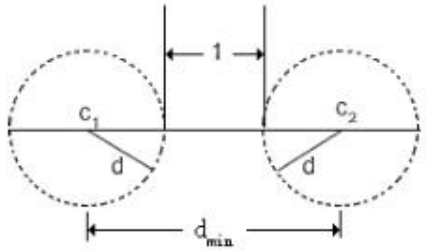
思考题

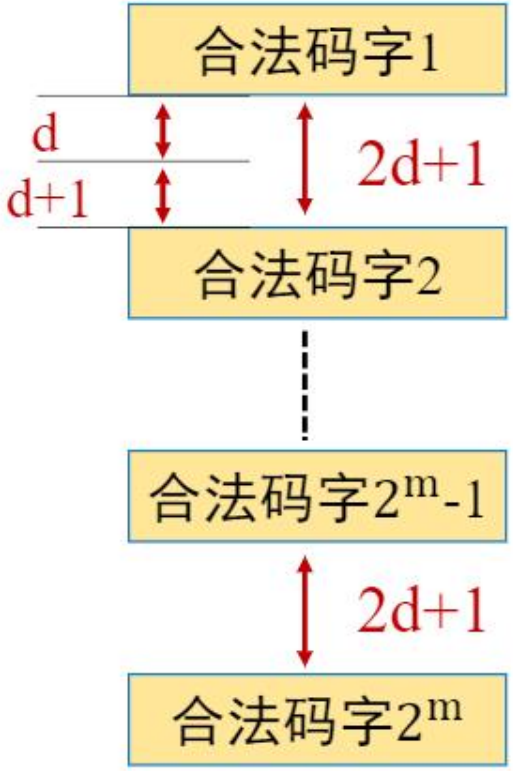
为了纠正d个错，可以使用海明距离为 2d+1 的编码?

思考题

为了纠正d个错，可以使用海明距离为 2d+1 的编码?

- 海明距离为2d+1的编码，能纠正d位差错。
  - 因为此时，如果一个码字有d位发生差错，它仍然距离原来的码字距离最近，可以直接恢复为该码。





差错检测和纠正概述

- 例如：
  - 一个只有4个有效码字的编码方案：
    0000000000, 0000011111, 1111100000, 1111111111
  - 海明距离为5，可以检测4位错，纠正2位错
  - 如果已知只有1位或2位错误，接收方接收0000000111
    - 则可知原码字为：0000011111
  - 如果发生3位错误，例如0000000000变成0000000111，接收方无法纠正错误，但可以检测出错误

- 随着海明距离的增加，纠错的能力也增加；即海明距离越大，纠错能力越强。
- 海明距离为3，可以纠正1个错误；而海明距离为5，才可以纠正2个错误。
- 当一个系统中的海明距离增加的时候，合法码字就减少了；即传输效率降低

检错：d+1
纠错：2d+1

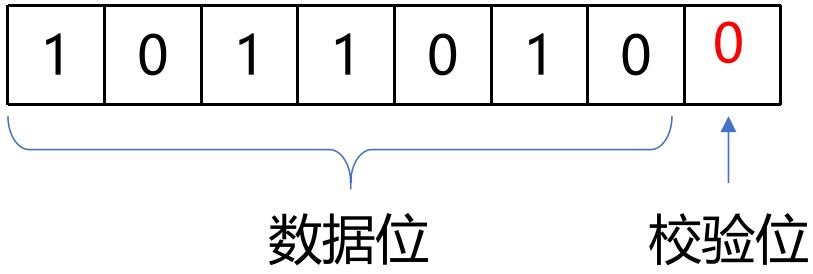
矛盾！

典型检错码

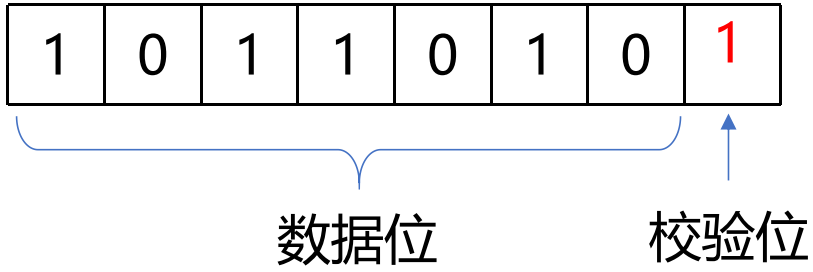
- 常用的检错码包括：
  - 奇偶检验 (Parity Check)：1位奇偶校验是最简单、最基础的检错码
  - 校验和 (Checksum)：主要用于TCP/IP体系中的网络层和传输层
  - 循环冗余校验 (Cyclic Redundancy Check，CRC)：数据链路层广泛使用的校验方法

典型检错码—奇偶校验

- 1位奇偶校验：增加1位校验位，可以检查奇数位错误
  - 偶校验：保证1的个数为偶数个，例如：



  - 奇校验：保证1的个数为奇数个，例如：



考虑二维奇偶校验，是否可以进行纠错?

典型检错码—校验和

➢ TCP/IP体系中主要采用的校验方法

发送方：

- 对每16bit进行二进制求和，如果和的高16bit不为0，则将和的高16bit和低16bit反复相加，直到和的高16bit为0，从而获得一个16bit的值；
- 将该16bit的值取反，存入校验和字段，一起发送

接收方：进行 16 位二进制补码求和运算（包含校验和），结果非全1，则检测到错误

数据

```
1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
---------------------------------
1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1
---------------------------------
1 0 1 1 1 0 1 1 1 0 1 1 1 0 0
```

校验和

```
0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1
```

数据

```
1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
---------------------------------
1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1
---------------------------------
0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1
---------------------------------
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

校验和

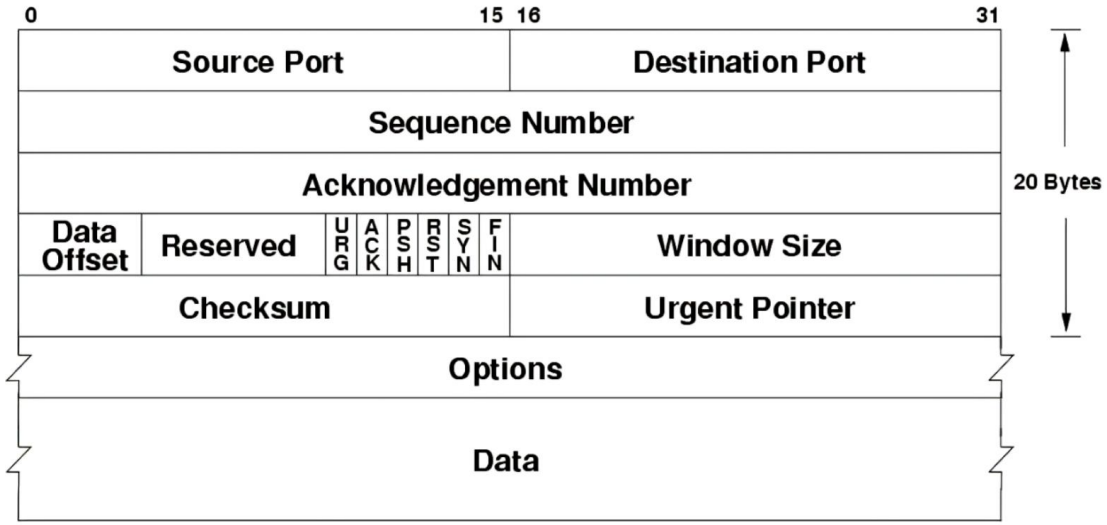
未检测到错误

典型检错码—校验和

- TCP/IP体系中主要采用的校验方法

IPv4 Header Format

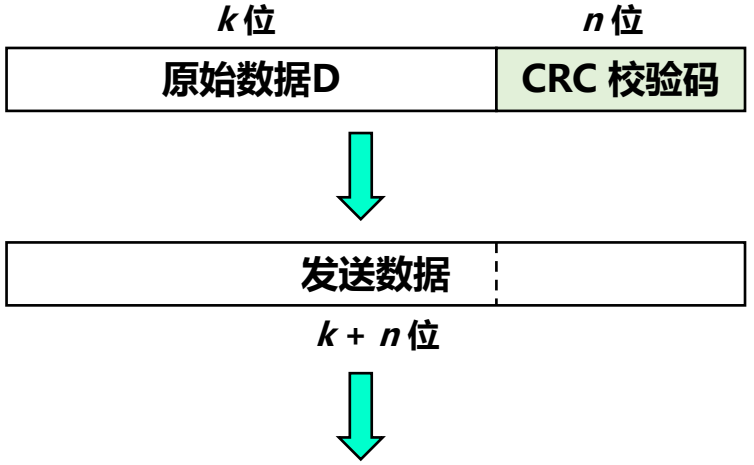
16 Bits			16 Bits			
Version (4 Bits)	Header Length (4 Bits)	Types of Service (8 Bits)	Total Length (16 Bits)			
Identification (16 Bits)			0	DF	MF	Fragment Offset (13 Bits)
Time to Live (8 Bits)	Protocol (8 Bits)		Header Checksum (16 Bits)			
Source IP Address (32 Bits)						
Destination IP Address (32 Bits)						
Options						



典型检错码—循环冗余校验CRC

➢ CRC校验码计算方法

- 设原始数据D为k位二进制位模式
- 如果要产生n位CRC校验码，事先选定一个n+1位二进制位模式G（称为生成多项式，收发双方提前商定），G的最高位为1
- 将原始数据D乘以2^n（相当于在D后面添加 n 个0），产生k+n位二进制位模式，用G对该位模式做模2除，得到余数R（n位，不足n位前面用0补齐）即为CRC校验码



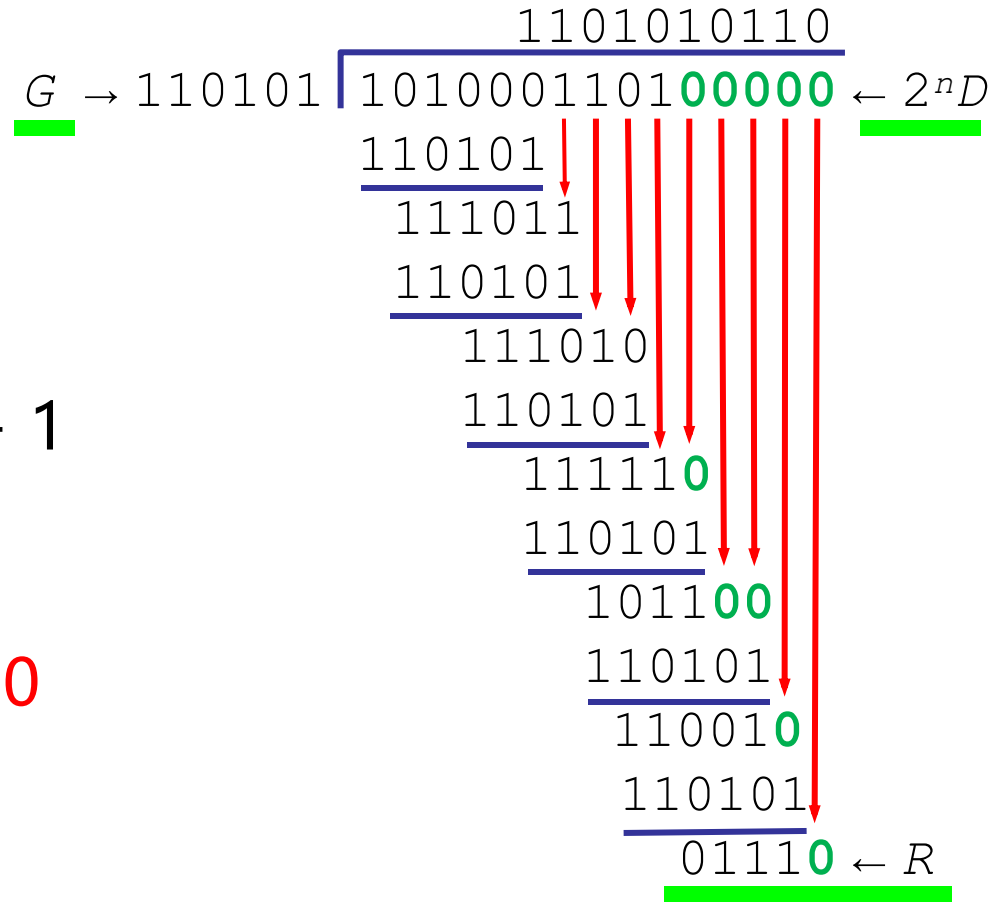
CRC校验能力：能检测出少于n+1位的突发错误

接收端如何计算？

检错码—循环冗余校验CRC

➢ CRC校验码计算示例

- D = 1010001101
- n = 5
- G = 110101 或 G = $x^5 + x^4 + x^2 + 1$
- R = 01110
- 实际传输数据：1010001101**01110**



检错码—循环冗余校验CRC

- 四个国际标准生成多项式
  - CRC-12 = $x^{12}+x^{11}+x^3+x^2+x+1$
  - CRC-16 = $x^{16}+x^{15}+x^2+1$
  - CRC-CCITT = $x^{16}+x^{12}+x^5+1$
  - CRC-32 = $x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$

例如，以太网、无线局域网使用CRC-32生成多项式

引自：清华大学崔勇老师提供的讲稿

- 特点：检错能力强，开销小，易于用编码器及检测电路实现。
- 检错能力：它所不能发现的错误的几率仅为0.0047%以下。
- 性能和开销：均远远优于奇偶校验及算术和校验等方式。
- 在数据存储和数据通讯领域CRC无处不在：

  - ✓ 著名的通讯协议X.25的FCS采用CRC-CCITT;
  - ✓ WinRAR、NERO、ARJ、LHA等压缩工具软件采用的是CRC-32;
  - ✓ 磁盘驱动器读写采用CRC-16;
  - ✓ 图像存储格式GIF、TIFF等也都用CRC作为检错手段。

思考：

循环冗余校验码，能够检查出多少位错误？

思考：

循环冗余校验码，能够检查出多少位错误？

- CRC的错误检测能力依赖于关键多项式的阶次以及所使用的特定关键多项式。
- 误码多项式E(x)。E(x)是接收到的消息码字与正确消息码字的异或结果。
- 当且仅当误码多项式能够被CRC多项式整除的时候CRC算法无法检查到错误。

课练：

如果生成多项式是 $G(x)= x^3 + x^2 + 1$，待传送的原始码字分别是 1111 和 1100，请计算采用CRC编码后的码字分别是多少？

课练：

如果生成多项式是 $G(x)= x^3 + x^2 + 1$，待传送的原始码字分别是1111 和 1100，请计算采用CRC编码后的码字分别是多少？

参考答案

1111 111

1100 101

课练:

解答（1）：生成多项式是3阶的，所以r=3，生成多项式对应的位（除数）是：1101

待传输的1111，移位后变为：1111000（被除数），得到余数111，用1111000−111，得到编码后的码字为：1111111

```
          1011
      ┌────────
1101  1111000
      1101
      ────
        1000
        1101
        ────
          1010
          1101
          ────
           111
```

课练:

解答（2）：生成多项式是3阶的，所以r=3，生成多项式对应的位（除数）是：1101

待传输的1100，移位后变为：1100000（被除数），得到余数101，用1100000-101，得到编码后的码字为：1100101

```
          1001
      ------------
1101 ) 1100000
       1101
       ----
         1000
         1101
         ----
          101
```

典型纠错码

- 设计纠错码
  - 要求：m个信息位，r个校验位，纠正单比特错

思考：要传输的数据有m位，冗余位r至少是多少，才能纠正1为错误?

引自：清华大学崔勇老师提供的讲稿

典型纠错码

- 设计纠错码
  - 要求：m个信息位，r个校验位，纠正单比特错
  - 对2^m个有效信息中任何一个，有n个与其距离为1的无效码字：
    - 可以考虑将该信息对应的合法码字的n位逐个取反，得到n个距离为1的非法码字，需要n+1个位模式来标识
  - 因此有：$(n + 1)\ 2^m \le 2^n$
  - 利用 $n = m + r$，得到 $(m + r + 1) \le 2^r$
  - 在给定m的情况下，利用该式可以得出纠正单比特错误校验位数的下界

课练：

r=1、2、3、4、5、6... 分别能纠正多长码字中的单个错误?

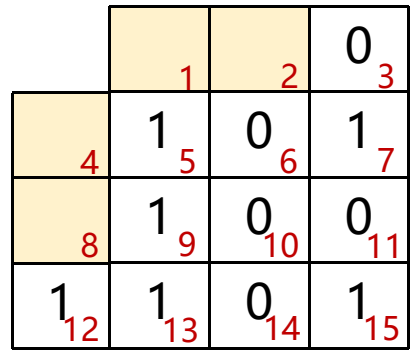
- 纠正单个错误需要的校验位的下界满足：

m	r	n（码字的总位数）
1	2	3
2~4	3	5~7
5~11	4	9~15
12~26	5	17~31
27~57	6	33~63
58~120	7	65~127

$$(m+r+1) \le 2^r$$

典型纠错码—海明码

- 目标：以奇偶校验为基础，如何找到出错位置，提供1位纠错能力
- 理解海明码编码过程，以 (15, 11)海明码为例
  - 例如：11比特的数据01011001101
  - 11比特数据按顺序放入数据位
  - 校验位：2的幂次方位（记为p1, p2, p4, p8）
  - 每个校验位对数据位的子集做校验，缩小定位错误的范围
  - 问题：每个校验位如何计算?
  - 海明码缺省为偶校验（也可以使用奇校验）



校验位　数据位

右下角数字为码字中位序号

典型纠错码—海明码

- 能纠一位错
- 编码形式：

$r_1\ r_2\ m_3\ r_4\ m_5\ m_6\ m_7\ r_8\ \ m_9\ m_{10}\ m_{11}\ \ldots$

- r为效验位，其下标为2的整次幂的位。
- m信息位，其下标为2的非整次幂的位。
- 信息位第k位，和相应2的整次幂的校验位有关。

例：（下标的二进制表达）

- m_3 和r_1，r_2 有关
- m_5 和r_1，r_4 有关
- m_6 和r_2，r_4有关
- m_7 和r_1，r_2，r_4有关

$$r_1 = m_3 \oplus m_5 \oplus m_7$$
$$r_2 = m_3 \oplus m_6 \oplus m_7$$
$$r_4 = m_5 \oplus m_6 \oplus m_7$$

例：对0010求汉明码。

解：

m_3	m_5	m_6	m_7
0	0	1	0

r_1	r_2	m_3	r_4	m_5	m_6	m_7
0	1	0	1	0	1	0

偶校验

校验位的计算(m=7，r=4)

	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11
	P1	P2	D1	P3	D2	D3	D4	P4	D5	D6	D7
$1=2^0$	√		√		√		√		√		√
$2=2^1$		√	√			√	√			√	√
$4=2^2$				√	√	√	√				
$8=2^3$								√	√	√	√

海明码实例之校验位计算

	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11
	P1	P2	D1	P3	D2	D3	D4	P4	D5	D6	D7
信息码	-	-	1	-	0	0	1	-	0	0	0
检验位	0	0	-	1	-	-	-	0	-	-	-
海明码	0	0	1	1	0	0	1	0	0	0	0

使用偶校验，一个校验集合里的1的个数是偶数

典型纠错码—海明码

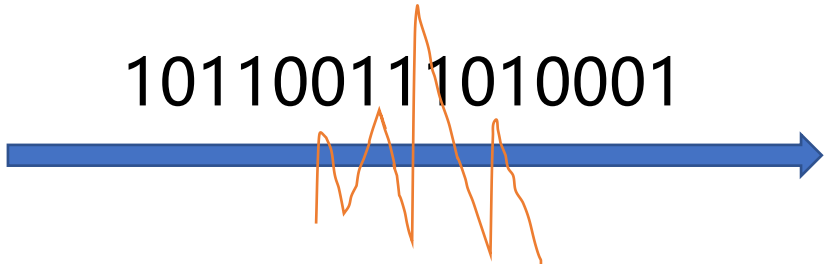
➢ 码字的发送与接收

- 如果发送过程中第7位出现差错，如何定位错误?

	1 1	**0** 2	0 3
1 4	1 5	0 6	1 7
0 8	1 9	0 10	0 11
1 12	1 13	0 14	1 15

校验位 数据位

发送方



	1 1	**0** 2	0 3
1 4	1 5	0 6	**0** 7
0 8	1 9	0 10	0 11
1 12	1 13	0 14	1 15

校验位 数据位

接收方

处理方式：

发方：

- 编码：r_i = 各相关信息位的异或

收方：

- 解码：设C=0，
  - r_i与相关信息位的异或为0，C+=0
  - r_i与相关信息位的异或为1，C+=i
  - C为出错位置。

例：若汉明码为1101101，判定是否有错，并错在何处？(写出原正确码)

$$r_1 \oplus m_3 \oplus m_5 \oplus m_7$$
$$r_2 \oplus m_3 \oplus m_6 \oplus m_7$$
$$r_4 \oplus m_5 \oplus m_6 \oplus m_7$$

例：若汉明码为1101101，判定是否有错，并错在何处？(写出原正确码)

$$r_1 \oplus m_3 \oplus m_5 \oplus m_7$$
$$r_2 \oplus m_3 \oplus m_6 \oplus m_7$$
$$r_4 \oplus m_5 \oplus m_6 \oplus m_7$$

解：r_1 r_2 m_3 r_4 m_5 m_6 m_7

1 1 0 1 1 0 1

- 初始 C=0
- i = 1，异或为1，C+= i , C=1
- i = 2，异或为0，C+= 0 , C=1+0=1
- i = 4，异或为1，C+= i , C=1+4=5
- 出错位为m_5位
- 即：原正确码为1101001

海明码的应用

- 用汉明码纠正非突发性错

  将总信息分为K组，每组进行汉明编码，共能纠正K位错。

- 用汉明码纠正突发性错

  将信息分为K行N列，对每行进行汉明编码，恢复长度最多为K的突发性错。

用海明码纠正突发性错：

- k个码字组成一个矩阵，一行一个码字(大小为n)，每行都采用海明码。
- 从左开始每次发送一列，帧到达接收方重组成矩阵。
- 如果有k位连续发生突发错误，则每个码字中只用一位发生错误，可用海明码纠正。

位传输顺序 ↓

m_{11}	m_{12}	……	m_{1n}
m_{21}	m_{22}	……	m_{2n}
……			
m_{k1}	m_{k2}	……	m_{kn}

例：

$$r_1 \oplus m_3 \oplus m_5 \oplus m_7$$
$$r_2 \oplus m_3 \oplus m_6 \oplus m_7$$
$$r_4 \oplus m_5 \oplus m_6 \oplus m_7$$

字符	ASCII		校验位
H	1001000	位传输顺序	00110010000
a	1100001		10111001001
m	1101101		11101010101
m	1101101		11101010101
i	1101001		01101011001
n	1101110		01101010110
g	1100111		11111001111

水平校验

垂直发送

典型纠错码—海明码（小结）

- 海明码纠正的实现过程
  - 每个码字到来前，接收方计数器清零
  - 接收方检查每个校验位k (k = 1, 2, 4 ...)的奇偶值是否正确（每组运算）
  - 若 pk 奇偶值不对，计数器加 k
  - 所有校验位检查完后，若计数器值为0，则码字有效；若计数器值为 j，则第 j 位出错。例：若校验位p1、p2、p8出错，则第11位变反
- 使用海明码纠正突发错误
  - 可采用k个码字（n = m + r）组成 k × n 矩阵，按列发送，接收方恢复成 k × n 矩阵
  - kr个校验位，km个数据位，可纠正最多为k个的突发性连续比特错

引自：清华大学崔勇老师提供的讲稿

本章内容

3.1 数据链路层的设计问题

3.2 差错检测和纠正

3.3 基本的数据链路层协议

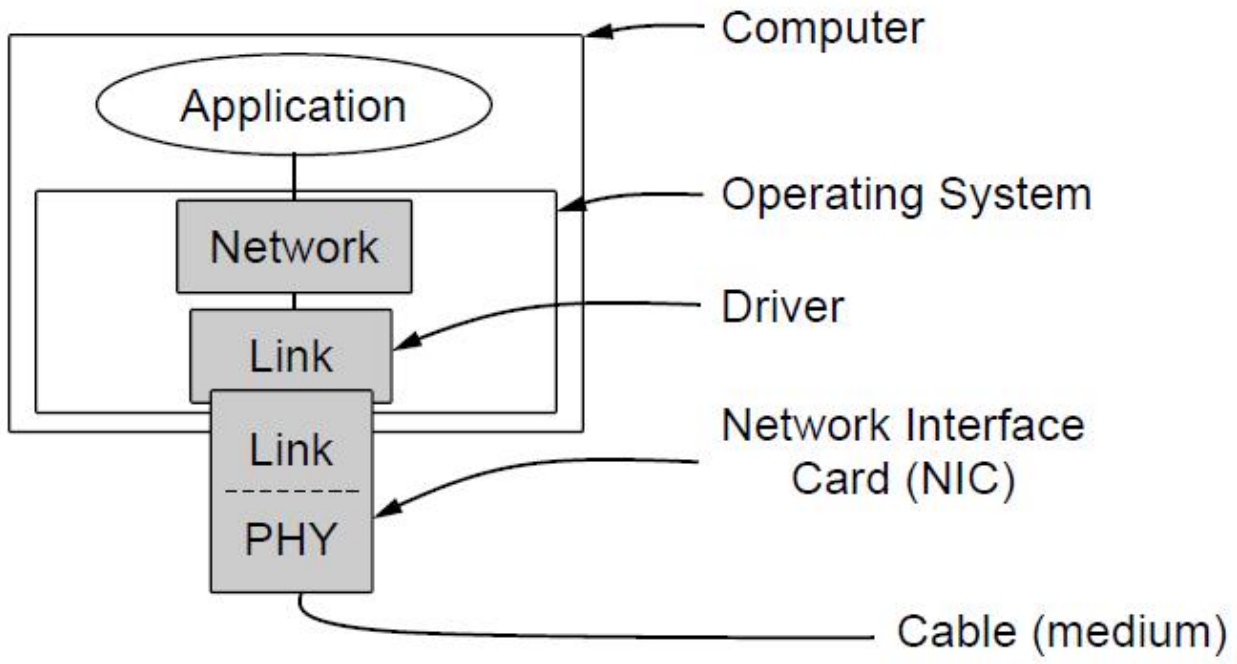
3.4 滑动窗口协议

3.5 数据链路协议实例

1. 定义与假设
2. 乌托邦式单工协议
3. 无错信道单工停止-等待协议
4. 有错信道单工停止-等待协议

物理层、数据链路层和网络层的实现

- 物理层进程和某些数据链路层进程运行在专用硬件上（网络接口卡）
- 数据链路层进程的其他部分和网络层进程作为操作系统的一部分运行在CPU上，数据链路层进程的软件通常以设备驱动的形式存在



基本数据链路协议

- 由浅入深地学习六个协议，了解数据链路层的主要工作，有些工作原理和概念是网络通信的基础，其运行机理在网络层和传输层也会采用。
- 首先介绍最简单的三个协议：
  - 无限制的单工协议
  - 单工停—等协议
  - 有噪声信道的单工协议

数据的传输在某时是单向的

3.3 停止等待协议

如何提高信道利用率？

- 全双工
- 捎带确认
- 批量发送数据：滑窗技术(协议4~协议6)
  - 协议4：n=1——引出滑动窗口的基本概念
  - 协议5：回退n帧（Go Back n）
  - 协议6：选择重传（Select Repeat）

3.4 滑动窗口协议

停止等待协议

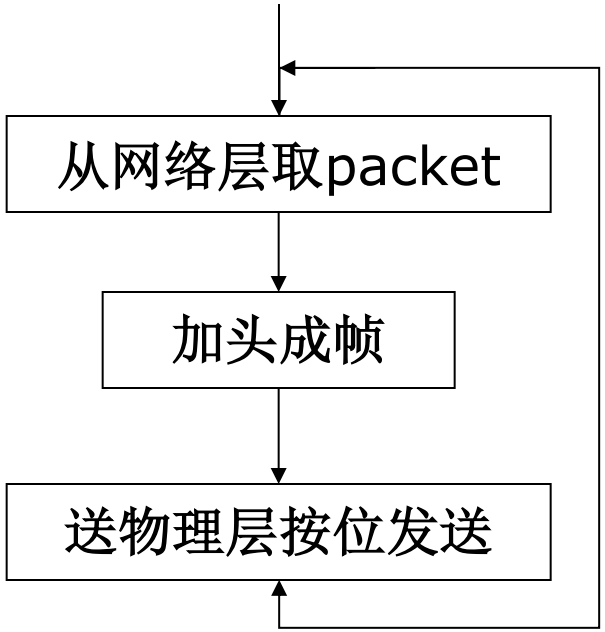
一．无限制单向数据传送协议(完全理想化)

条件：

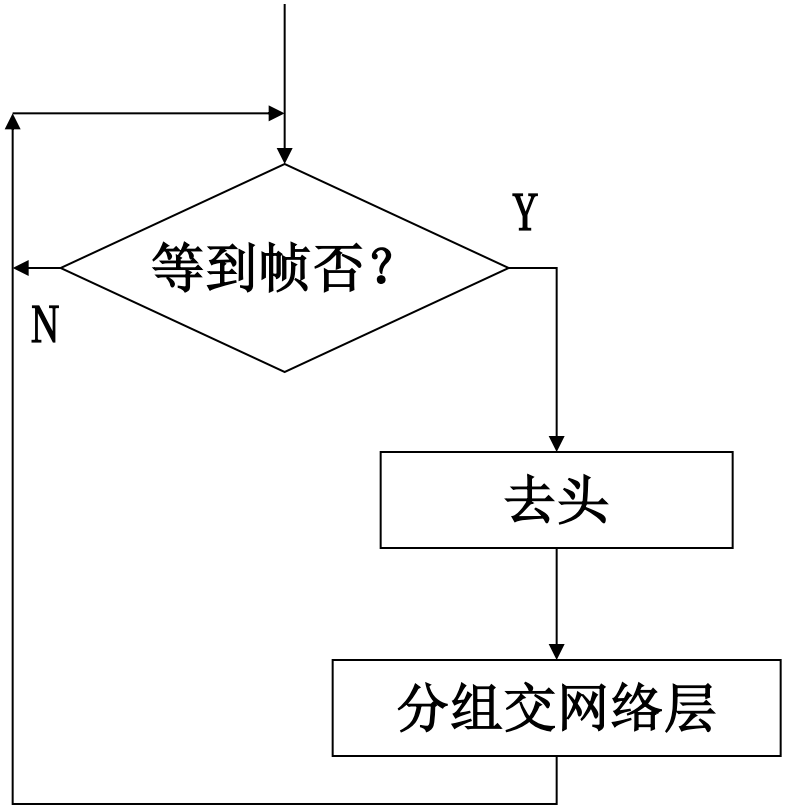
1. 一个机器发数据，另一机器收数据；
2. 网络层总是准备好分组传送；
3. 通道无错；
4. DLL层处理速度无穷大。

流程图：

发：



收：

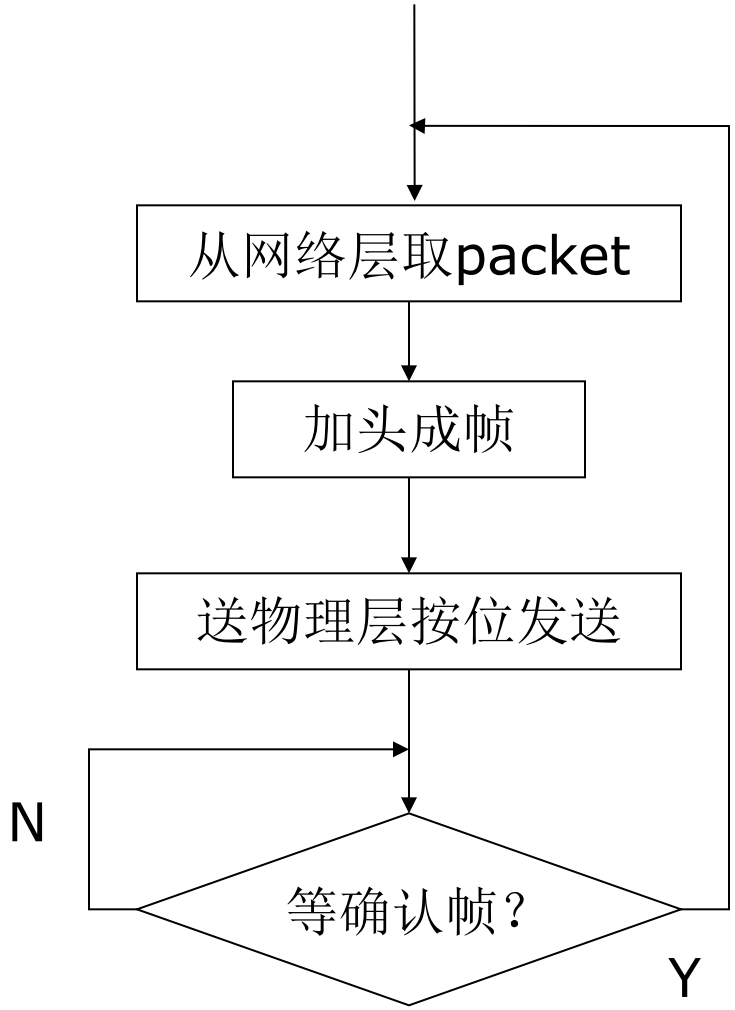


二．单向停-等协议

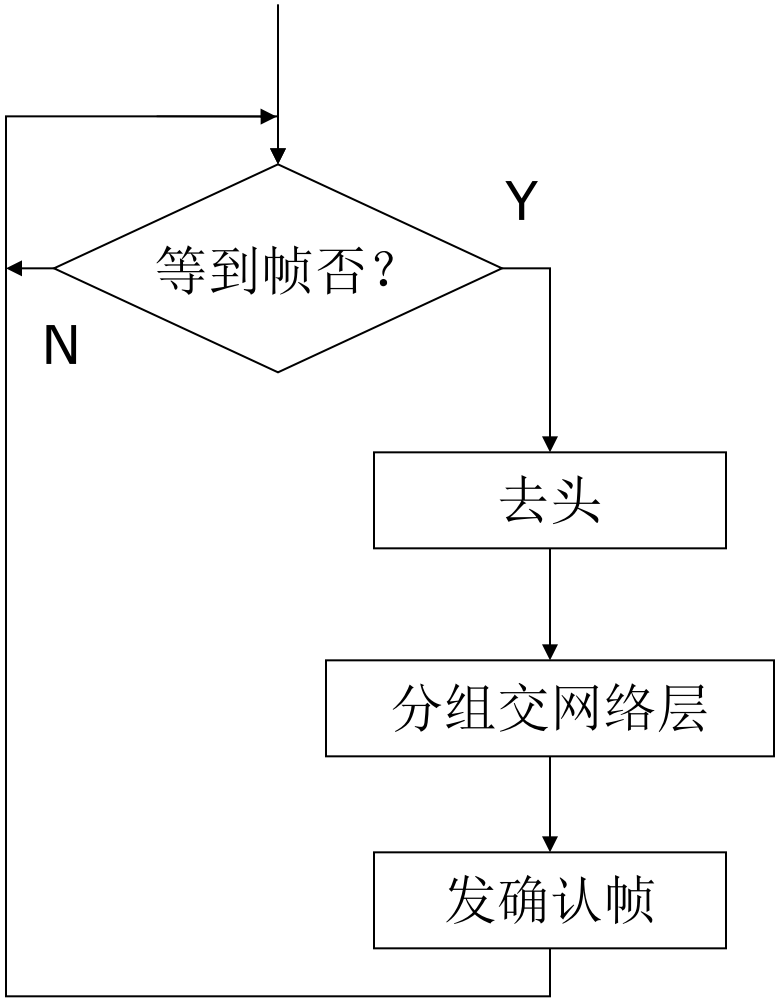
条件：

1. 一个机器发数据，另一机器收数据
2. 网络层总是准备好分组传送
3. 通道无错
4. DLL层处理速度有限

发：



收：



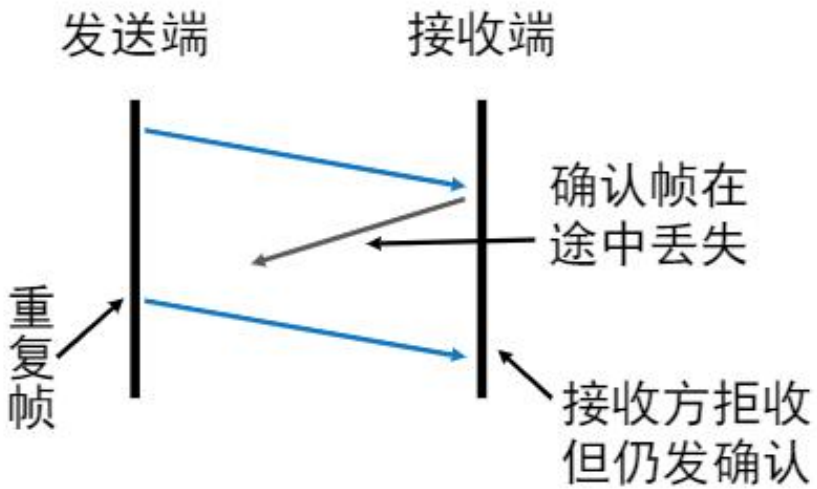
三、有错，单向数据传送协议

条件：

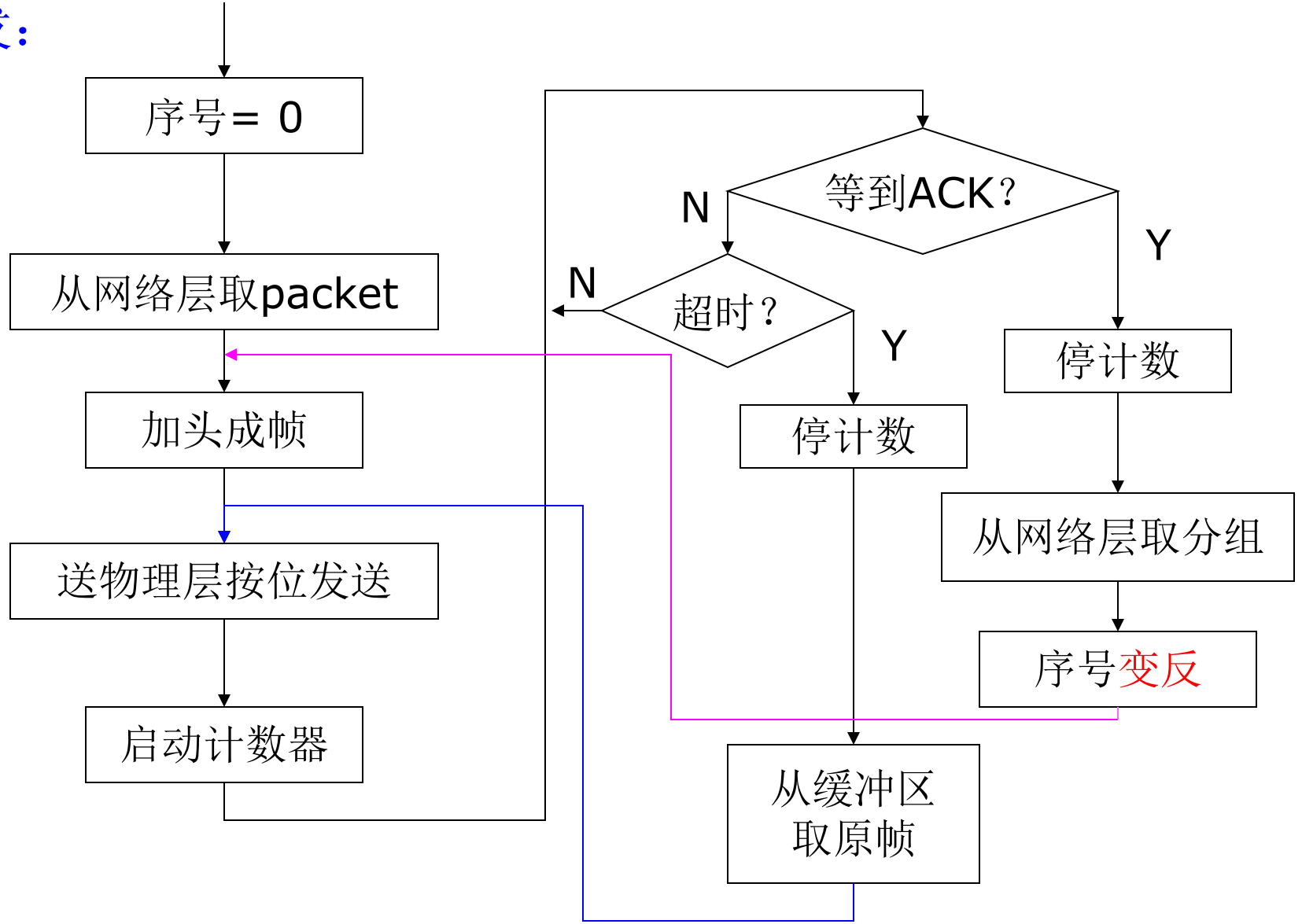
1. 一个机器发数据，另一机器收数据；
2. 网络层总是准备好分组传送；
3. 通道有错；
4. DLL层处理速度有限。

- 有噪声就会产生差错，有差错就可能会引起以下这些问题：
  - 数据帧或确认帧在途中丢失将如何解决？
    - 定时器超期：重传

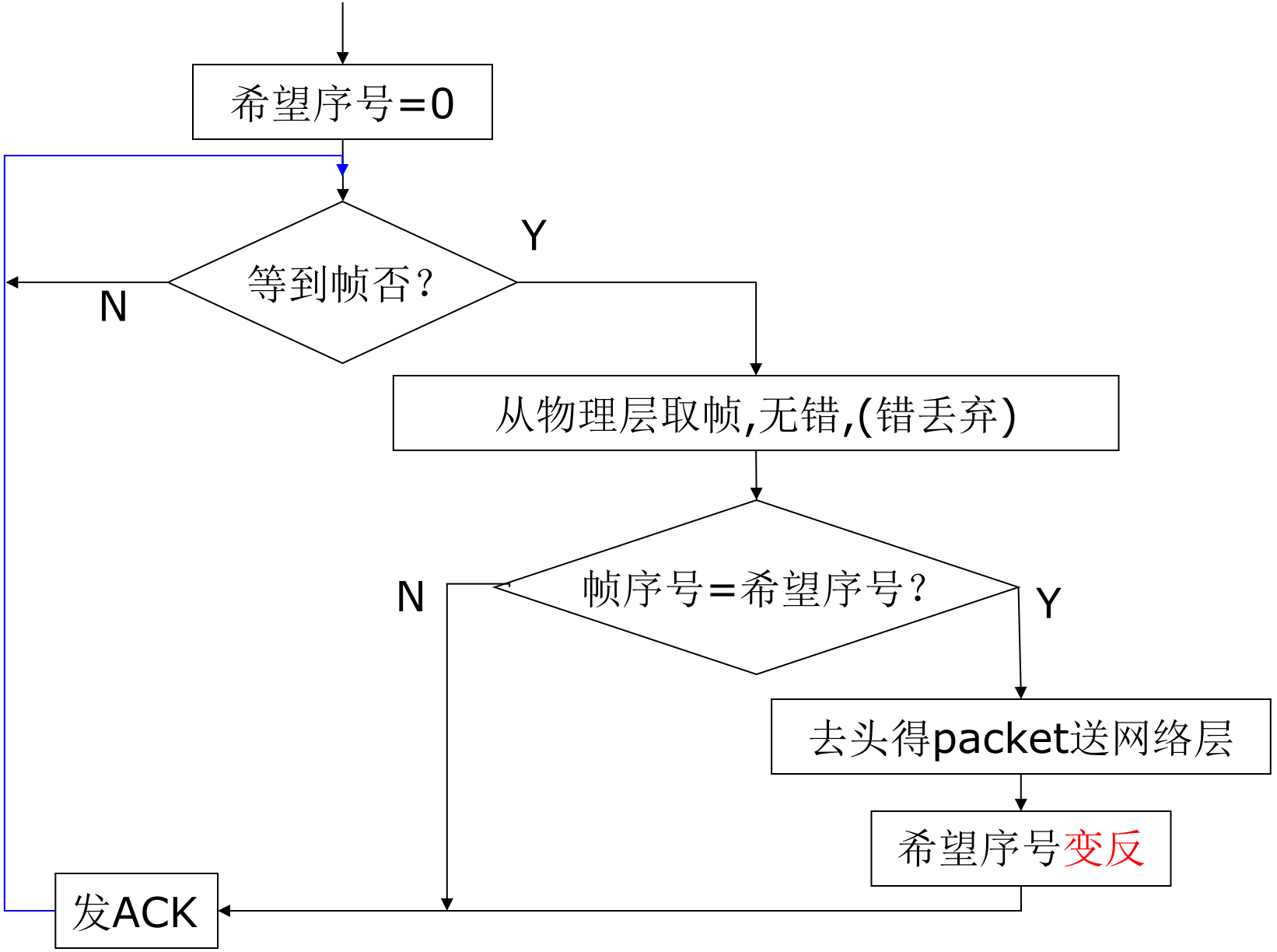
- 有噪声就会产生差错，有差错就可能会引起以下这些问题：
  - 有可能收到重复帧，如何解决？
    - 给每个帧一个独一无二的序列号
    - 序号也用来重组排序



发：


序号= 0
从网络层取packet
加头成帧
送物理层按位发送
启动计数器
等到ACK?
N
Y
超时？
N
Y
停计数
停计数
从网络层取分组
序号变反
从缓冲区
取原帧

收：


希望序号=0
等到帧否？
Y
N
从物理层取帧,无错,(错丢弃)
帧序号=希望序号？
N
Y
去头得packet送网络层
希望序号变反
发ACK

流程说明：

- 此图中没有表示出当收方又到一个错帧，向发方回应NAK帧，
- 计数器为了解决死锁问题，重发时间选择重要，一般选为略大于“从发完数据帧到收到回答帧所需的平均时间”。
- 重复帧或。应答帧丢失，造成收方收到两个相同的帧。
- 超时重传时间设置太短

  解决办法：对发送的每一帧加序号。
- 本协议只需一位做序号。

假设A向B发数据：

B在收到错误序号的数据帧时，除应丢弃这个错误序号的数据帧外，还必须重复发送前面已经发送过的最后一个确认帧。

由此可见，确认帧必须表明这样两个含义：

①第$n-1$号帧和这以前的各帧都收到了。

②我期望收到第n号帧。确认帧还可能包含这样的含义。

当A和B都有数据帧向对方发送时，确认信息可以放在数据帧中捎带地发送给对方。

数据传输的效率很低，怎么办？

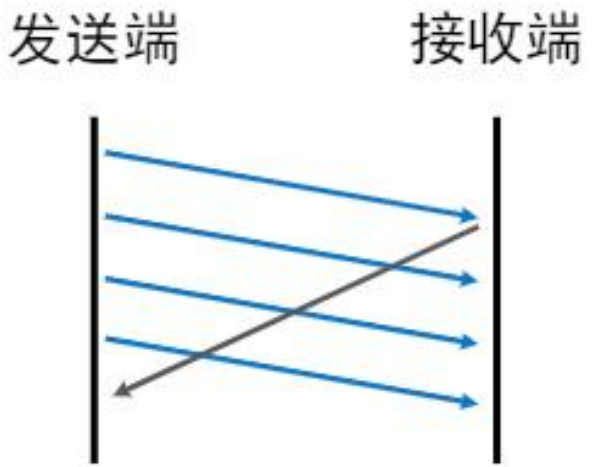
- 全双工
- 捎带确认
- 批发数据
- ？

捎带确认（piggyBacking）

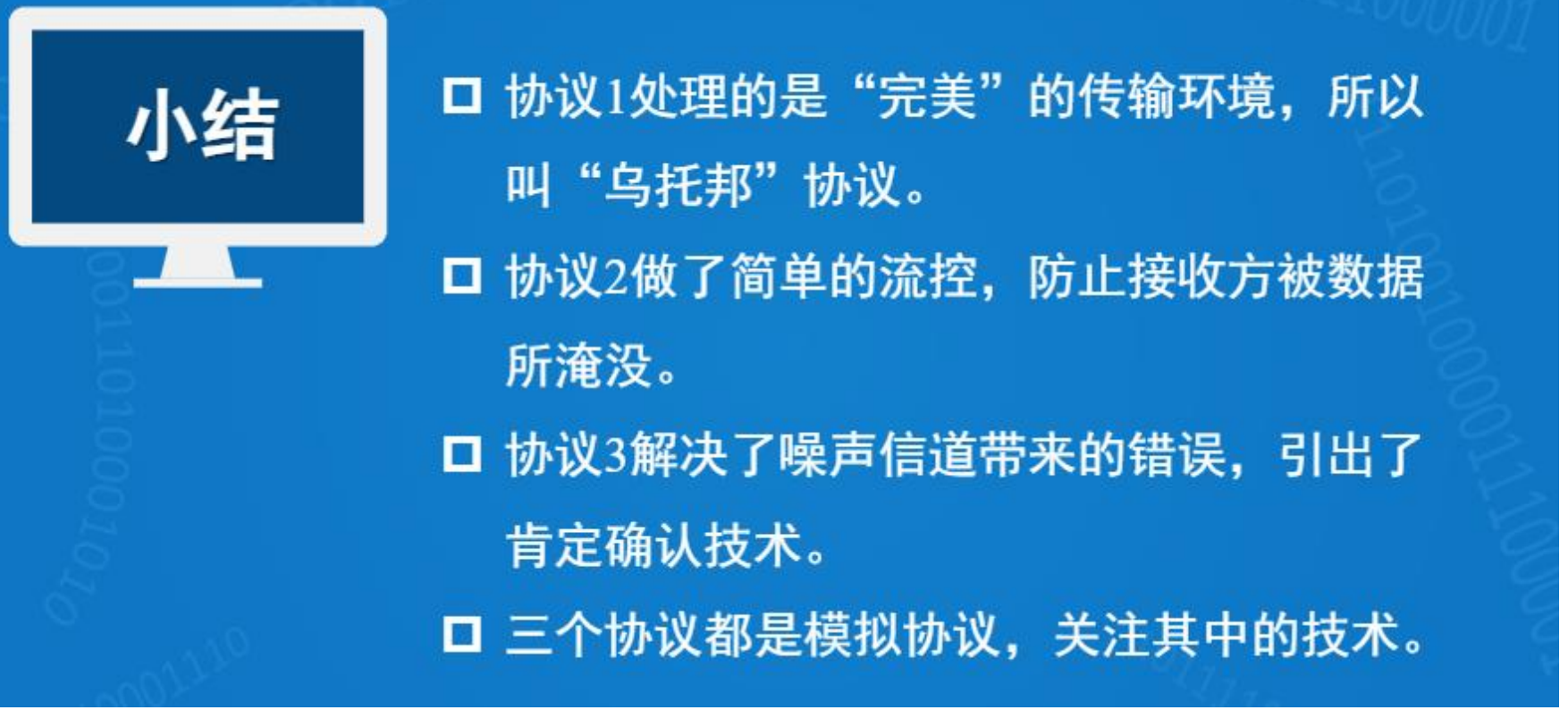
- 捎带确认的作用
  - 进一步减少确认帧
- 捎带确认：将确认暂时延迟以便可以钩到一个外发的数据帧（s. ack）
- 如无法“捎带”，当一个控制捎带确认的计时器超时后，单独发确认帧

批量发送数据

- 停—等协议是每收到一个确认才能发送下一帧，发送端等待时间太长，网络通信效率不高。为了提高效率，可以在等待的时间发送数据帧，这样大大减少了浪费的时间。



- 无限制的单工协议
- 单工停—等协议
- 有噪声信道的单工协议



本章内容

3.1 数据链路层的设计问题

3.2 差错检测和纠正

3.3 基本的数据链路层协议

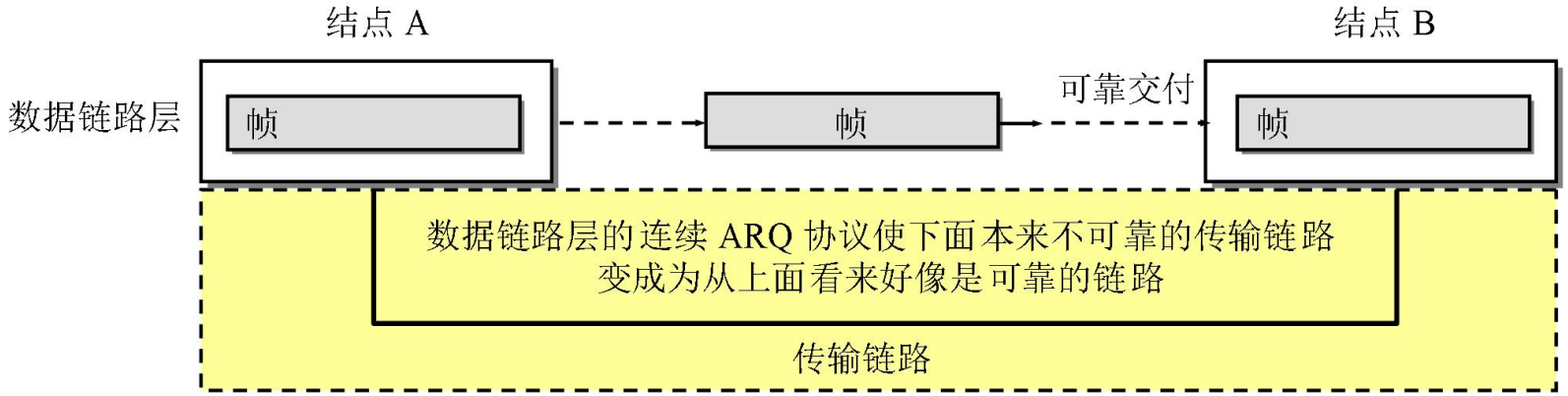
3.4 滑动窗口协议

3.5 数据链路协议实例

▪Automatic Repeat reQuest

ARQ协议的主要功能：

1. 在不可靠的链路上实现可靠交付(无差错、无丢失、无重复、按顺序)。所有的重传对上面都是透明的.



2. **支持了流量控制**。

当接收端来不及接收时，就可以通过暂时不发送确认帧来降低发送端发送数据帧的速率。

滑动窗口协议

一．条件：

1．双向数据传送；

2．网络层总是准备好分组发送；

3．有错通道；

4．DLL层处理速度有限 。

如何提高信道利用率？

- 全双工
- 捎带确认
- 批量发送数据：滑窗技术(协议4~协议6)
  - 协议4：n=1——引出滑动窗口的基本概念
  - 协议5：回退n帧（Go Back n）
  - 协议6：选择重传（Select Repeat）

二、窗口 (任何一端均有两个窗口)

1．发送窗口(W_T)：存放已发出但未确认的帧序号

1) 发帧(帧存入缓冲区)，上界加1

2) 收到窗口下界ACK信息，下界加1

3) 收到窗口外的ACK信息，丢掉

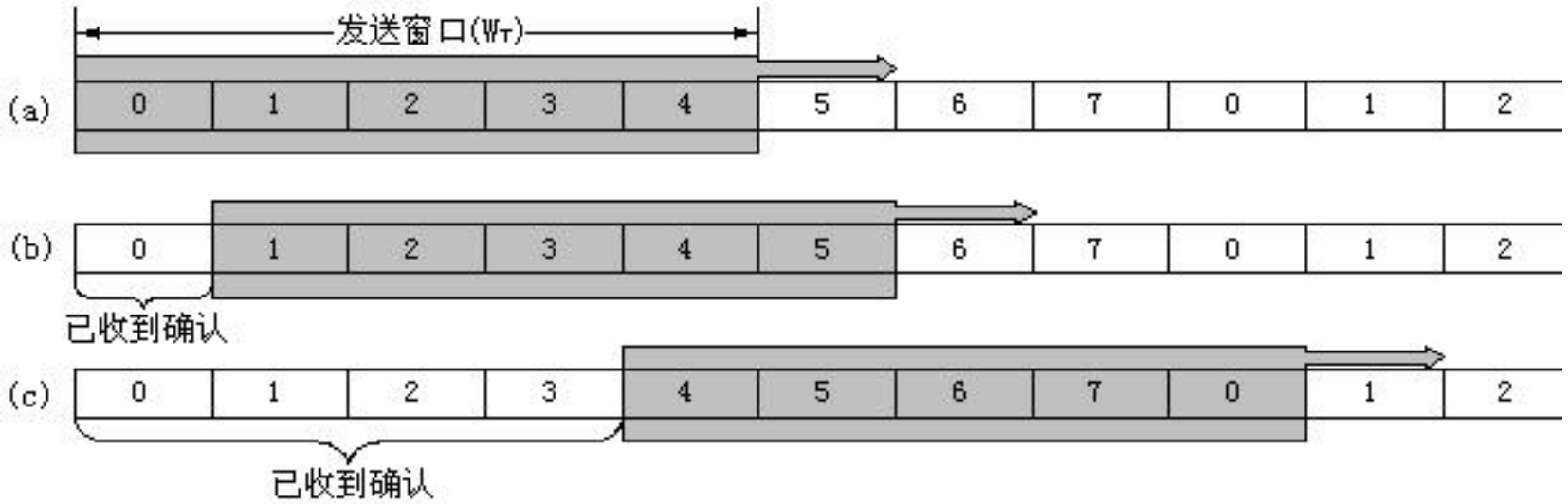
2．接收窗口(W_R)：存放应【期望】接收的帧序号

1) 收到接收窗口外的序号帧，丢掉

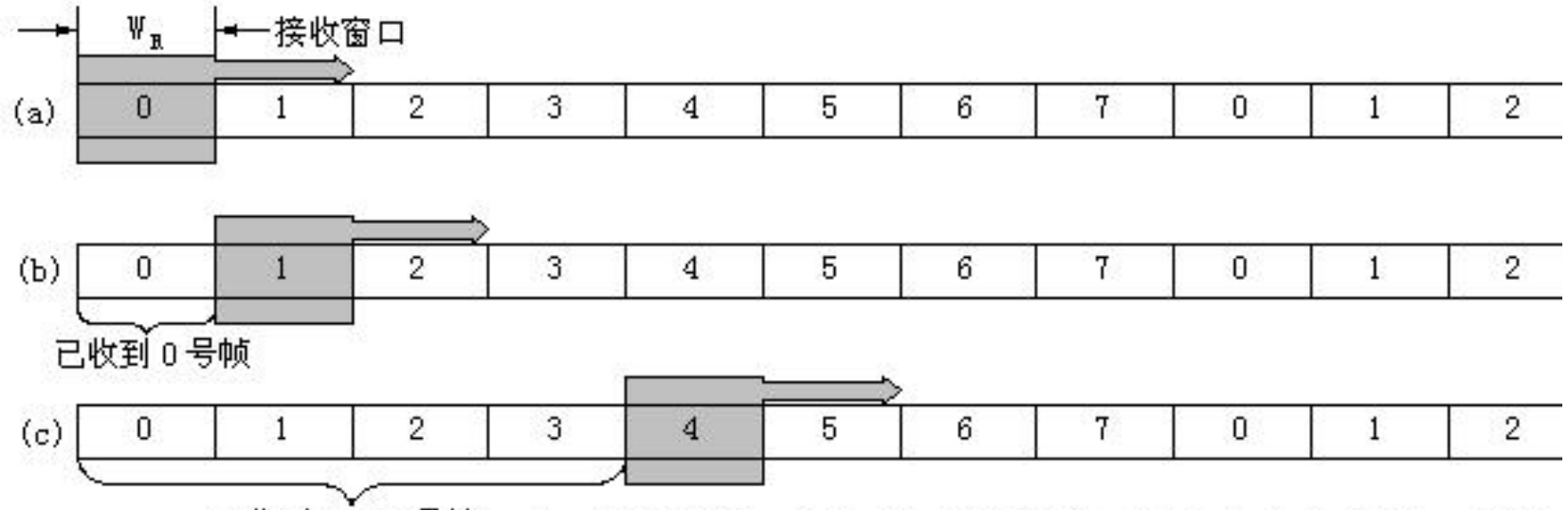
2) 收到接收窗口下界序号帧，接收窗口向前移一步

滑动窗口的基本概念

- 每个待发送帧被赋予一个序列号seq
  - seq的取值范围是 $0 \sim 2^n-1$（n位字段）
- 建立缓冲区
  - 发送窗口：缓存已发送、待确认的帧
    - 顺序接收来自网络层的分组，成帧，赋予序列号
    - 最多保存W个已经发送、等待确认的帧
    - 窗口达到最大值W时强制关闭网络层
  - 接收窗口：缓存期待接收的帧（序号）
    - 对进入窗口的帧顺序提交网络层，产生确认
    - 落在窗口外的帧被丢弃



（a）允许发送 0 到 4 号帧 （b）允许发送 1 到 5 帧 （c）允许发送 4 到 0 号帧



（a）准备接收 0 号帧 （b）准备接收 1 号帧 （c）准备接收 4 号帧

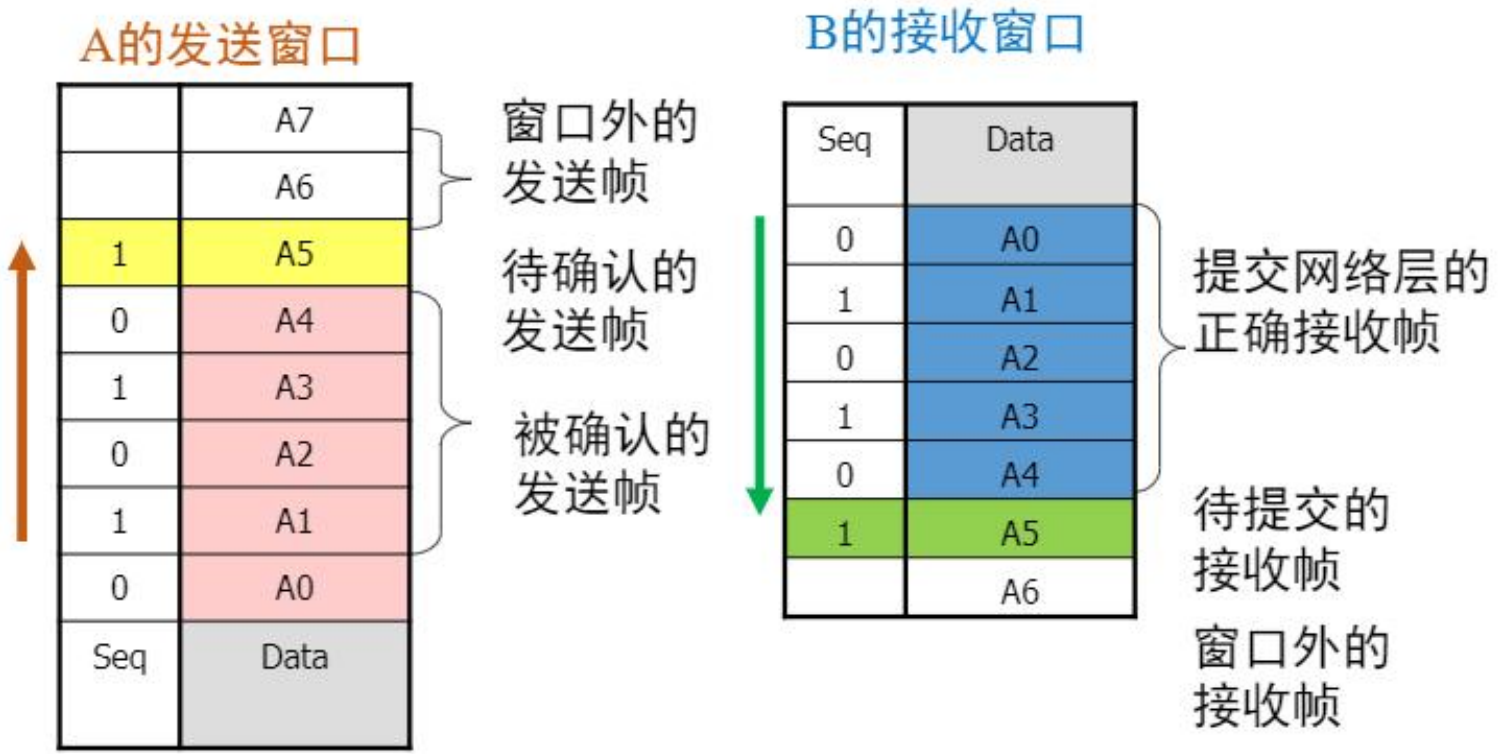
三种滑动窗口协议

1、一位滑动窗口协议(停止等待协议)【协议4】

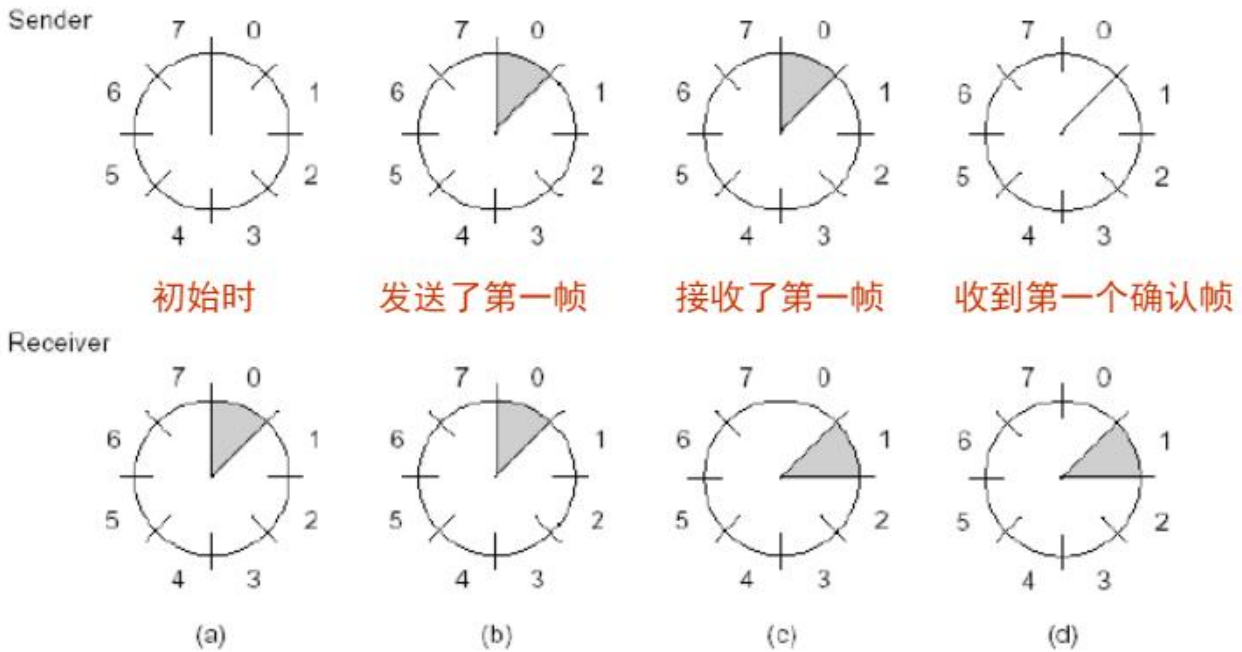
- 发送窗口(W_T)=1，
- 接收窗口(W_R)=1，

双向数据传送：

w=1滑动窗口概念


A的发送窗口
B的接收窗口
A7
A6
1
A5
0
A4
1
A3
0
A2
1
A1
0
A0
Seq
Data
窗口外的
发送帧
待确认的
发送帧
被确认的
发送帧
Seq
Data
0
A0
1
A1
0
A2
1
A3
0
A4
1
A5
A6
提交网络层的
正确接收帧
待提交的
接收帧
窗口外的
接收帧

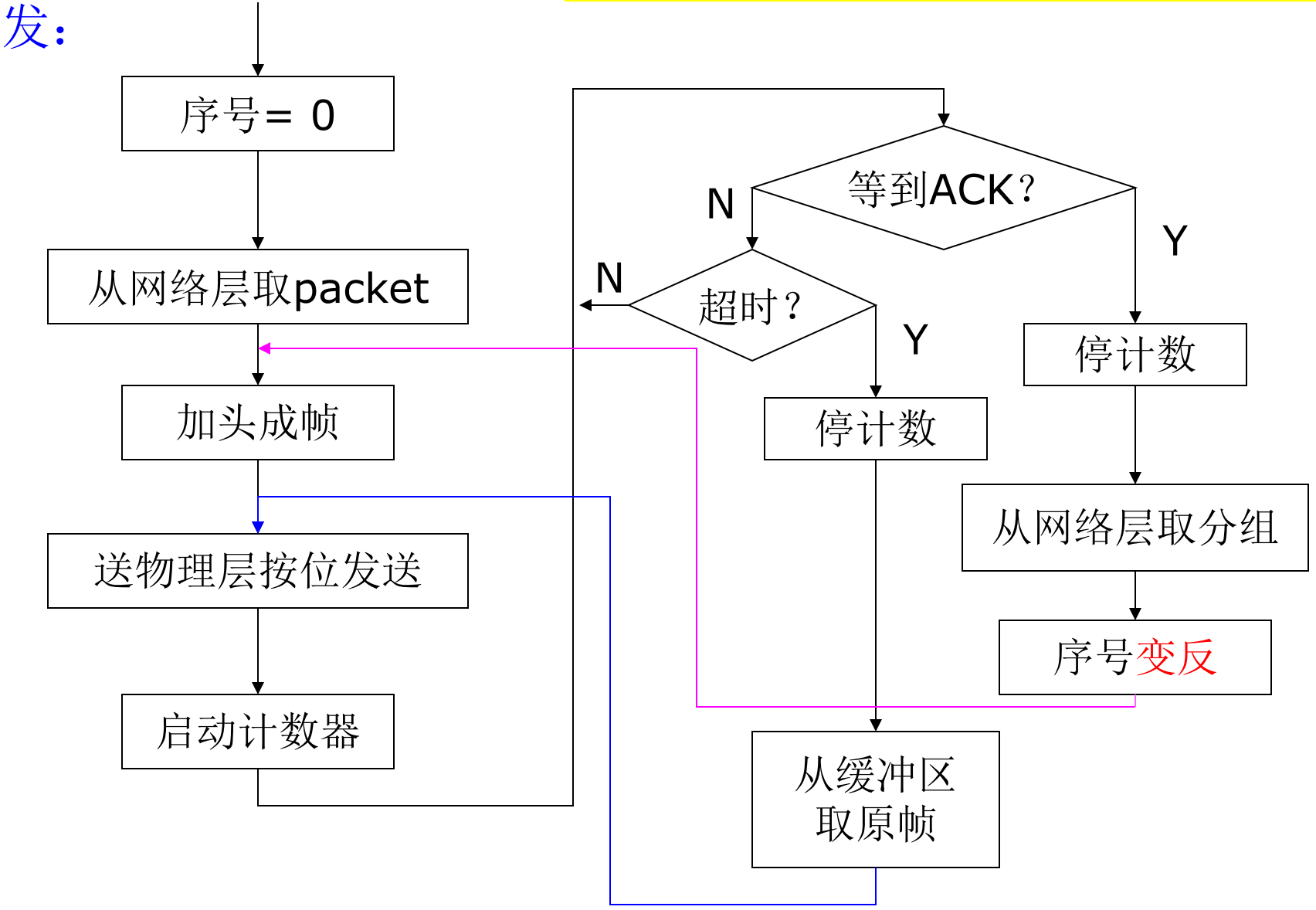
窗口大小为1，有3位序列号的滑动窗口



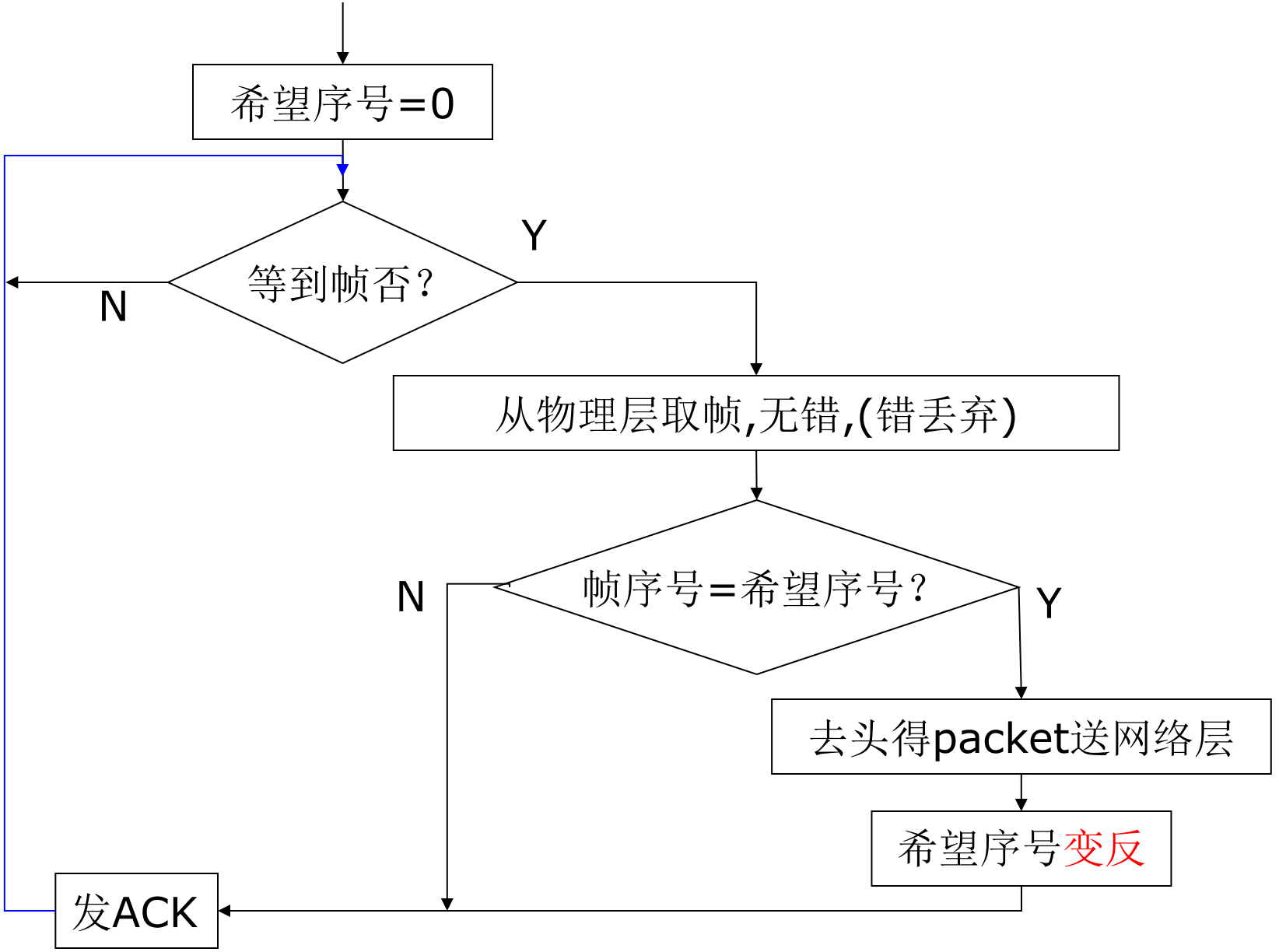
- 接收方收到帧后，首先核对是否为预期帧号(frame_expected)，如果是的，则接收并frame_expected+1，即移动接收窗口。
- 发送端收到应答帧，核对响应帧号next_frame_to_send，核对无误后，从网络层取新的帧，并执行next_frame_to_send+1，即移动发送窗口。如核对帧号不正确，则不移动窗口

停等协议可看作是发送和接收窗口均为1的滑窗

发：

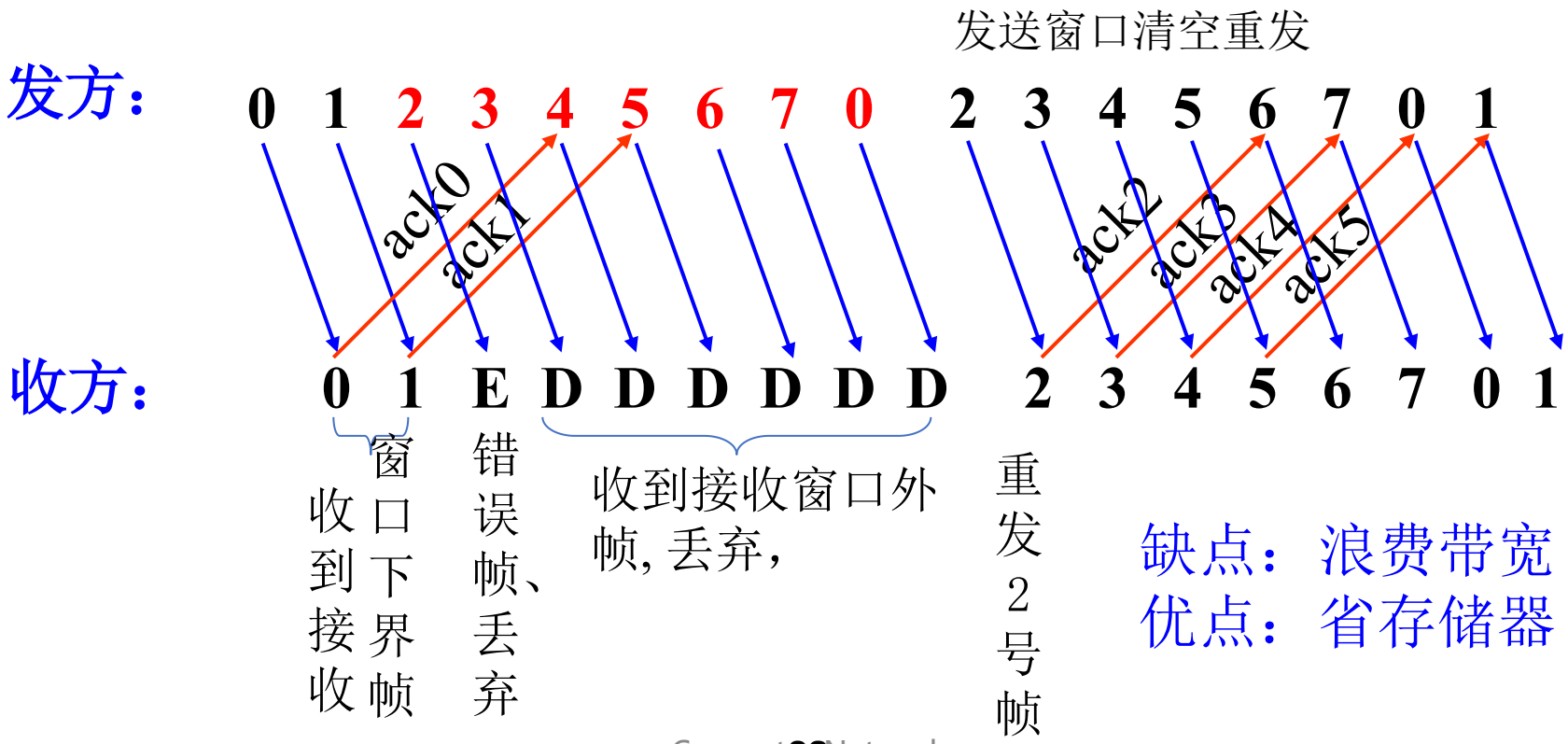


收：


希望序号=0
等到帧否？
Y
N
从物理层取帧,无错,(错丢弃)
帧序号＝希望序号？
N
Y
去头得packet送网络层
希望序号变反
发ACK

2、连续ARQ协议(又称Go-Back-N)【协议5】

- Automatic Repeat reQuest
- 发送窗口≥1，接收窗口≥1　　$W_T=2^n-1$时，$W_R=1$
- $W_T+W_R\leq 2^n$ 成立 (原因见教材后习题)
- 例：设n=3，$W_T=7$，$W_R=1$



协议5：回退n帧

- 接收方的接收策略选择：

  丢弃错帧，其后续帧因不是期望接收帧也被丢弃（接收窗口为1）

- 发送方的重传策略选择：

  缓存在发送窗口中的出错帧以及其后续帧全部重发

回退n帧协议的基本概念

- 定义序列号seq的取值范围和滑动窗口长度W
- 发送方连续发送至发送窗口满
- 接收窗口为1，对出错帧不确认（引发超时）
- 发送方超时重传，从未被确认帧开始

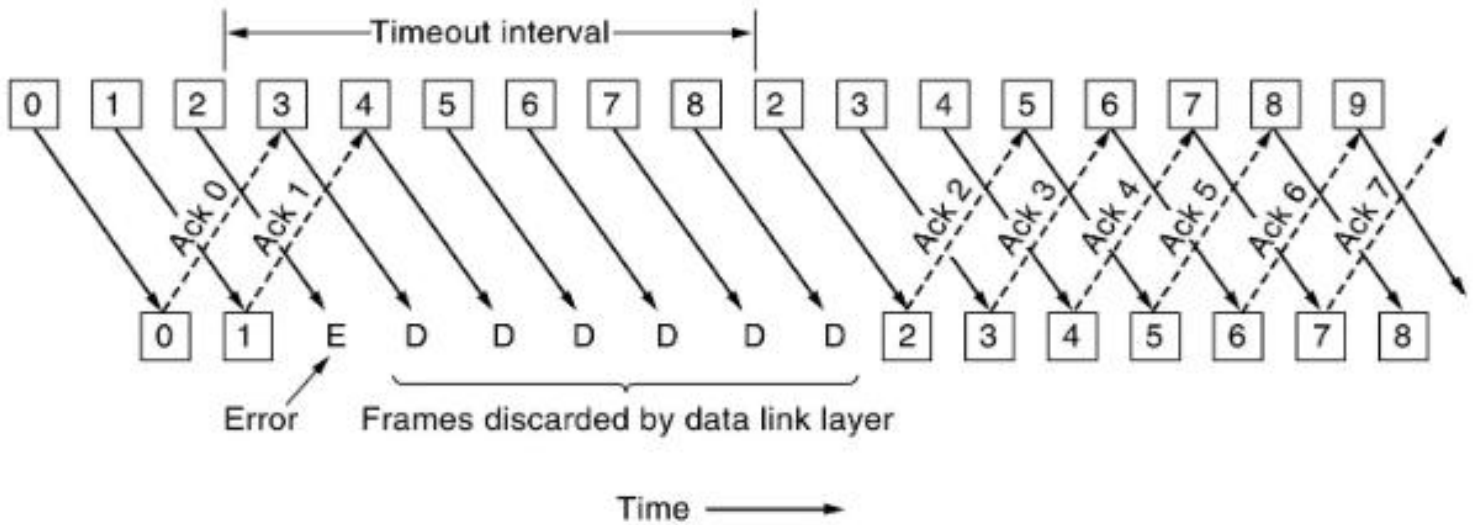
举例（MAX_SEQ = 7)

采用了累计确认

- 发送方一直保存着未被确认的帧。
- 累计确认
  - n号帧的确认到达时，暗含一个意思：n-1、n-2……等n号帧之前的帧也被确认
  - 当采用了累计确认，收到了n帧的确认，n帧之前的帧都可从缓存中删掉。

协议5回退n帧小结

- 接收窗口w=1
- 接收方遇到出错帧，怎么办？（D&ack）
- 发送方超时，重传出错帧及后续帧



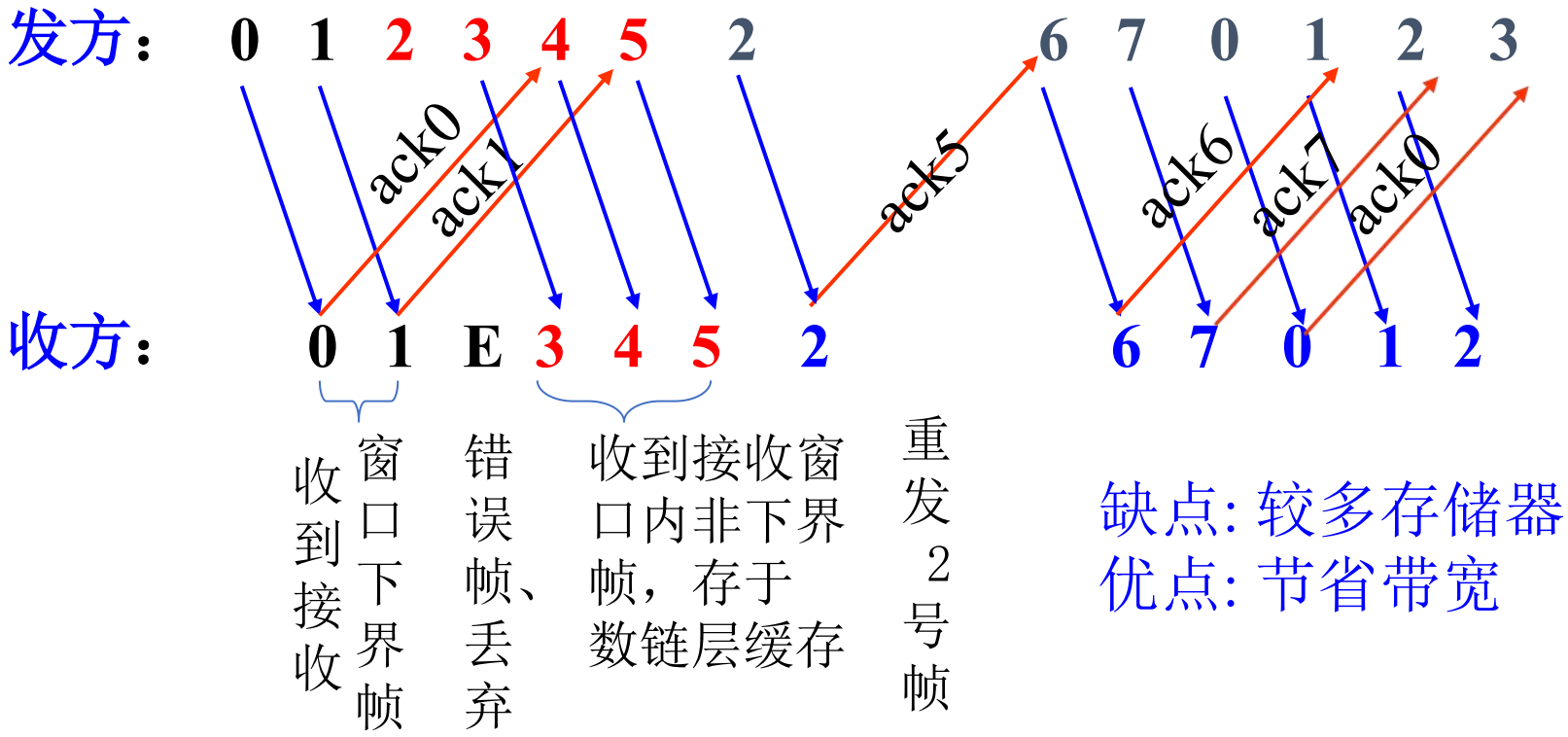
用管道化技术发送帧面临的新问题

- 出错情况
  - 连续发送W个数据帧，其中有一帧出错，但其后续帧被成功发送
- 接收方的接收策略选择
  - 丢弃错帧及后续帧，其后续帧因不是期望接收帧也被丢弃
  - 丢弃错帧，缓存后续正确接收帧
- 对应的发送方的重传策略选择
  - 缓存在发送窗口中的出错帧以及其后续帧全部重发——协议5
  - 只重发出错帧——协议6

3．选择重传ARQ协议(SR)【协议6】

- 发送窗口>1，接收窗口>1 **满足：**1）$W_T \le 2^{n-1}$ $W_R \le 2^{n-1}$；2) $W_T + W_R \le 2^n$, $W_T \le W_R$。W_T最大时有$W_T=W_R=(2^n)/2$ (见习题)

例：设n=3，$W_T=4$，$W_R=4$



缺点: 较多存储器
优点: 节省带宽

协议6：选择重传

- 接收方的接收策略选择：

  丢弃错帧，缓存后续正确接收帧；

- 发送方的重传策略选择：

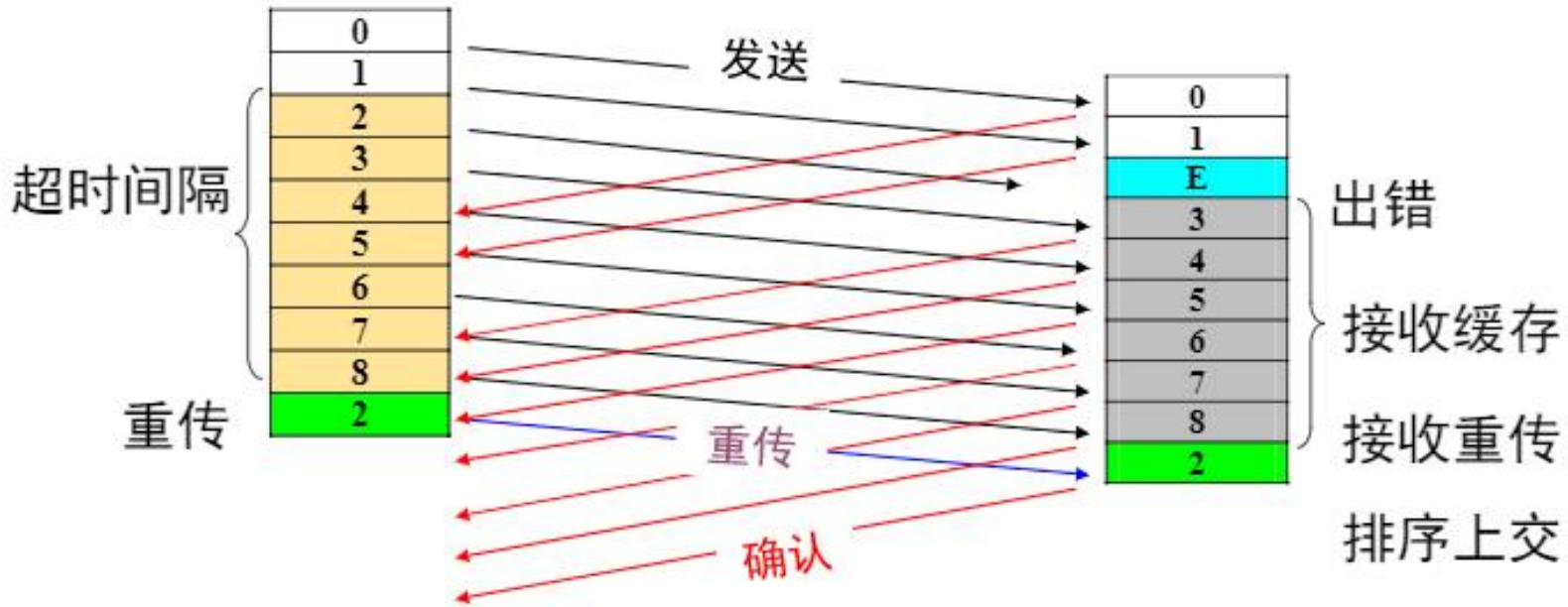
  只重发出错帧。

选择重传协议的基本概念

- 接收窗口存储差错帧后继的所有正确帧
- 发送方只重传差错帧
- 接收方接收重传帧，按正确顺序将分组提交网络层

举例（MAX_SEQ = 15 ）

例：选择重传协议



Seq=0~15；W=8

选择重传协议的工作原理分析

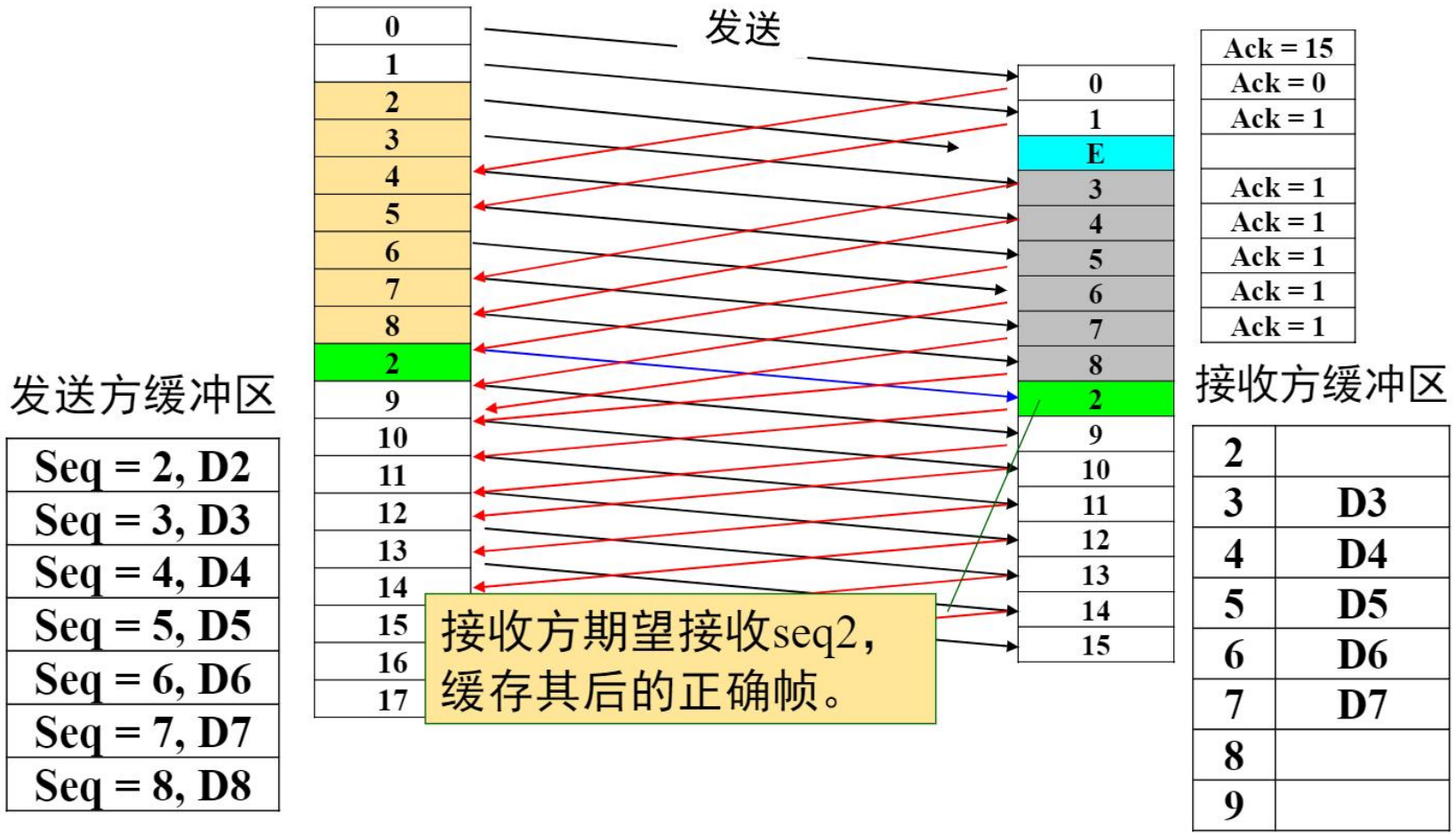
发送方

- 正常发送
  - 对帧编号，待确认帧缓存
- 收到确认
  - 释放确认帧所占缓冲区，滑动发送窗口
- 差错帧超时时间到
  - 重传缓存的最后被确认帧以后的那一帧

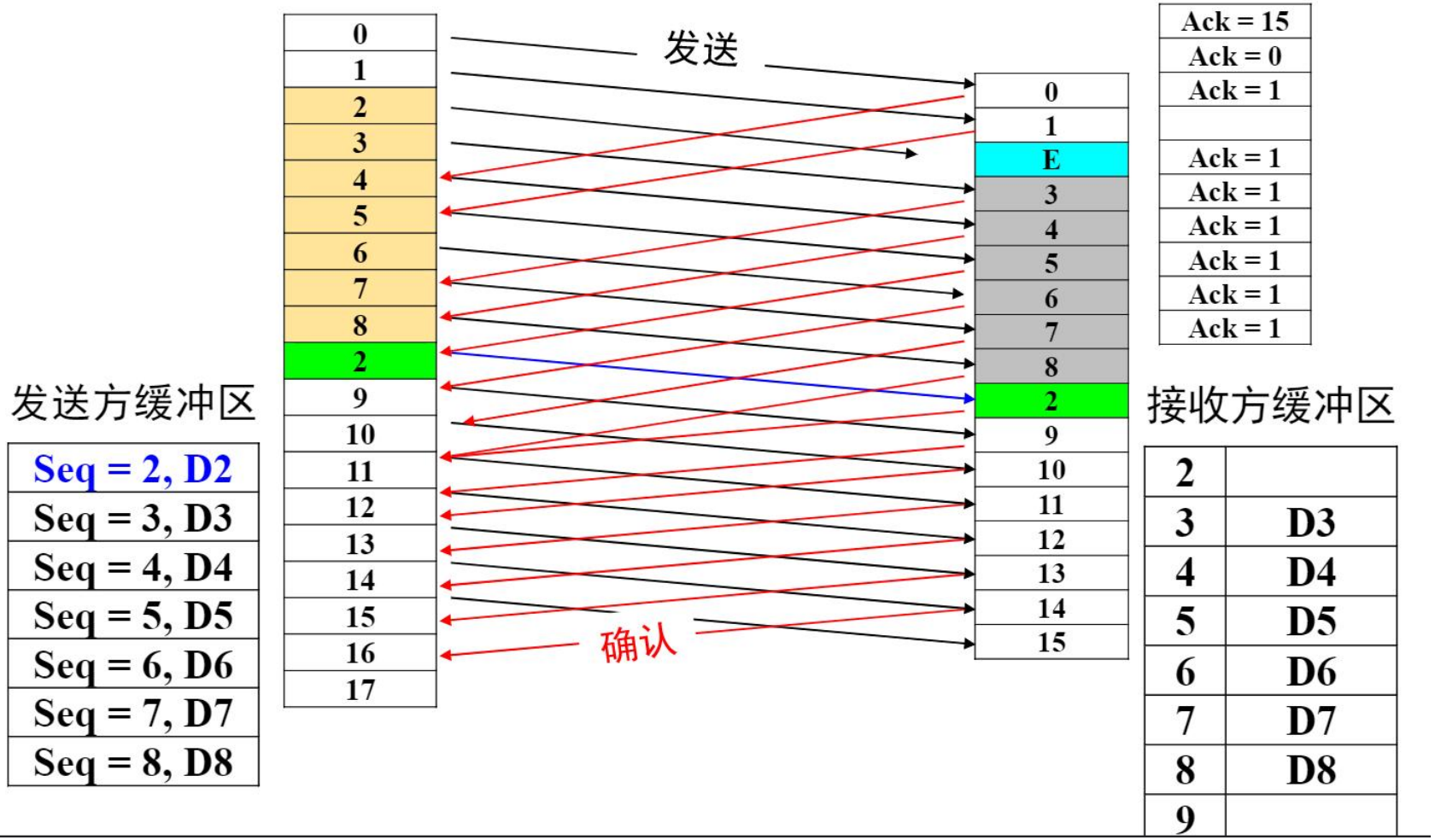
接收方

- 正常接收
  - 上交网络层、回送确认，滑动接收窗口
- 收到非期望的正确帧
  - 缓存，回送对接收的最后正确帧的确认
- 收到重传帧
  - 将缓存帧排序上交，回送确认，滑动接收窗口

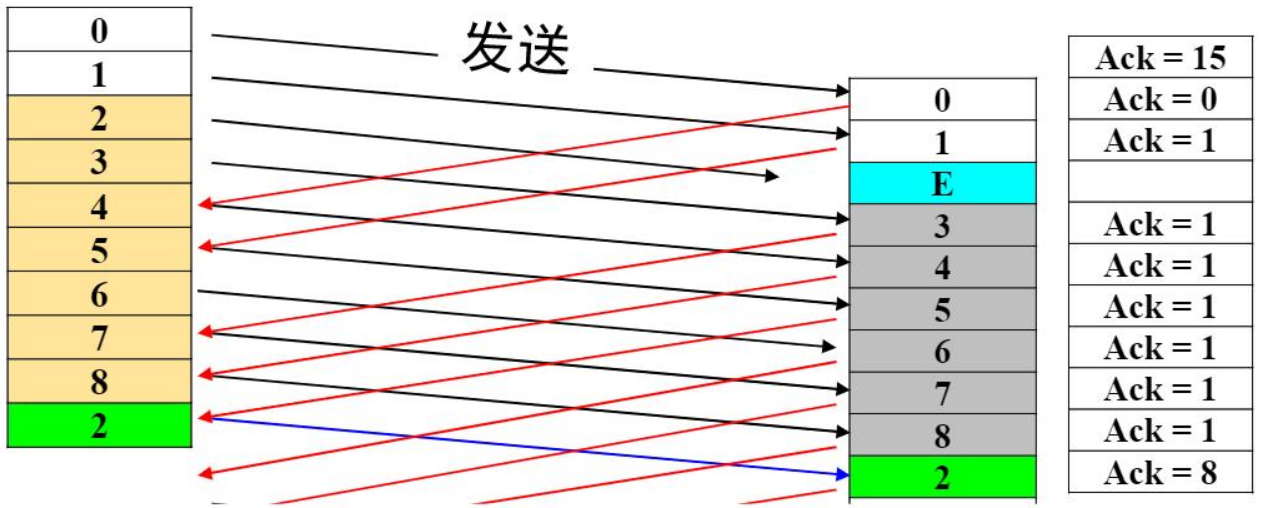
关键步骤：接收方收到非期望的正确帧—缓存



关键步骤：发送方选择帧seq2重传



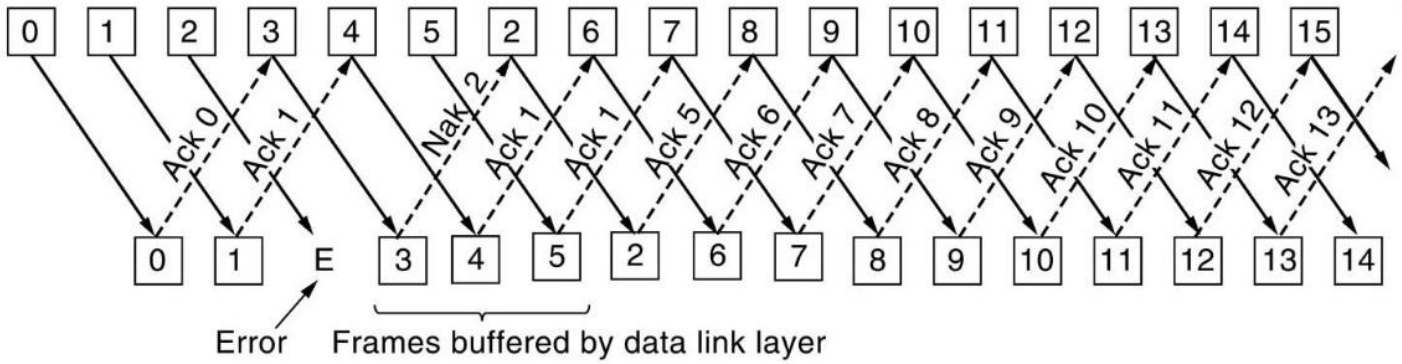
关键步骤：接收方收到重传帧seq2—排序上交



将帧seq2和接收方缓冲区中的帧正确排序，提交网络层，回送ack=8，滑动接收窗口。

协议6选择重传小结

- 接收方缓存
- 乱序帧，需正确排序



注意：NAK/累计确认的作用

- 否定确认NAK
  - 加快出错帧的重传
  - 对出错帧回送否定确认，使发方不再等到超时再重传

滑动窗口长度w的选择

- 协议6（选择重传）
  - MAX_SEQ =7（ Seq=0～ MAX_SEQ ）
  - W = 4
  - 接收窗口： W= (MAX_SEQ + 1) / 2
  - 发送窗口小于接收窗口

协议性能(不考虑丢包情况下的近似分析)

- 在协议4中假设：以下时间是可以忽略的：
  - 接收方处理到达帧的时间
- 事实上，在低速信道上，来回时间（RTT：the round-trip time）可能非常大 ，发送方在这段时间处于blocked状态

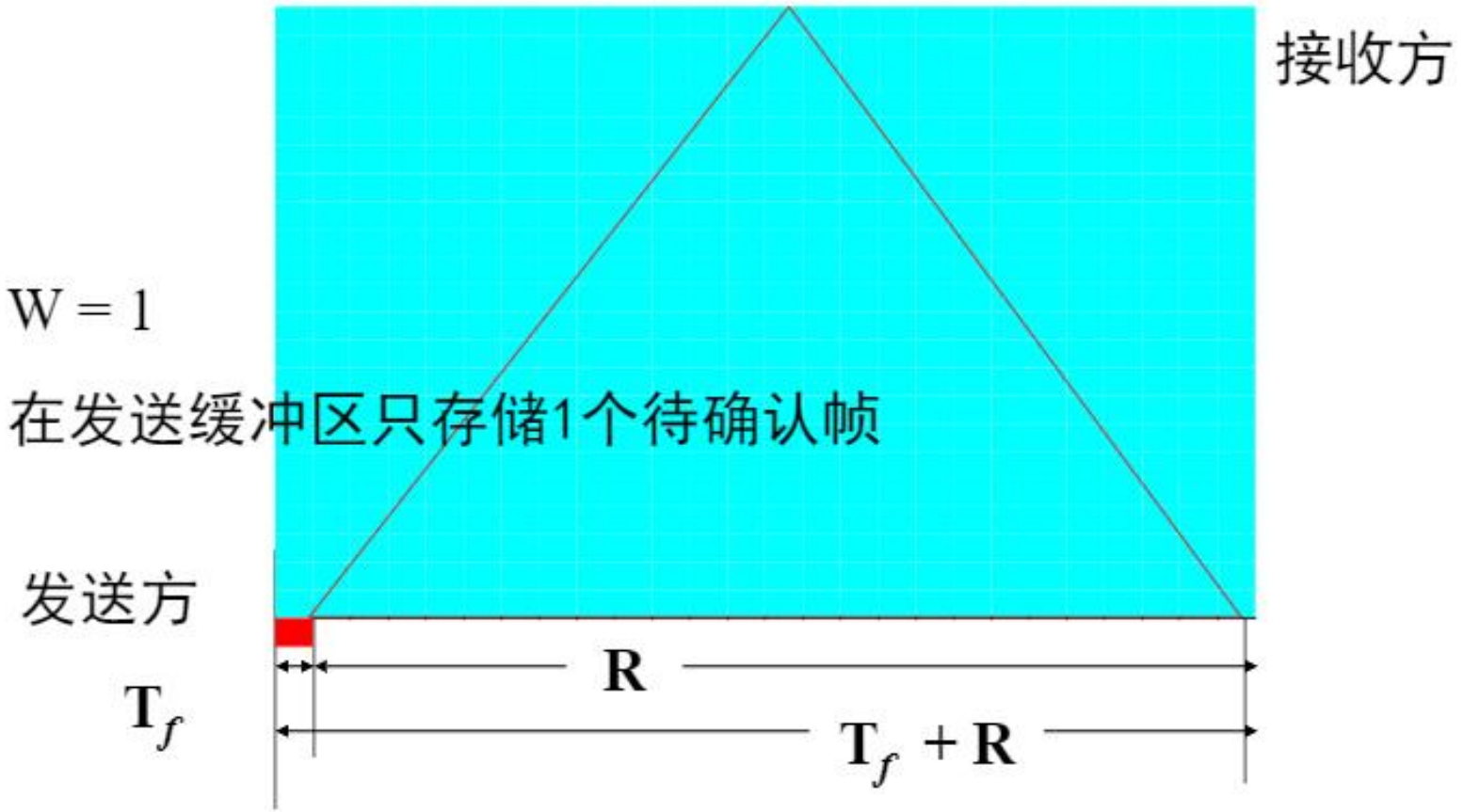
- 设数据传输率Bbps，帧长Lbit，信号在信道中来回平均传播时延是Rsec。
- 信道的实际有效利用率：

$$U = \frac{\frac{L}{B}}{\frac{L}{B} + R} = \frac{L}{L + BR}$$

例：某卫星信道50Kbps，帧长1000bit，R=500ms。

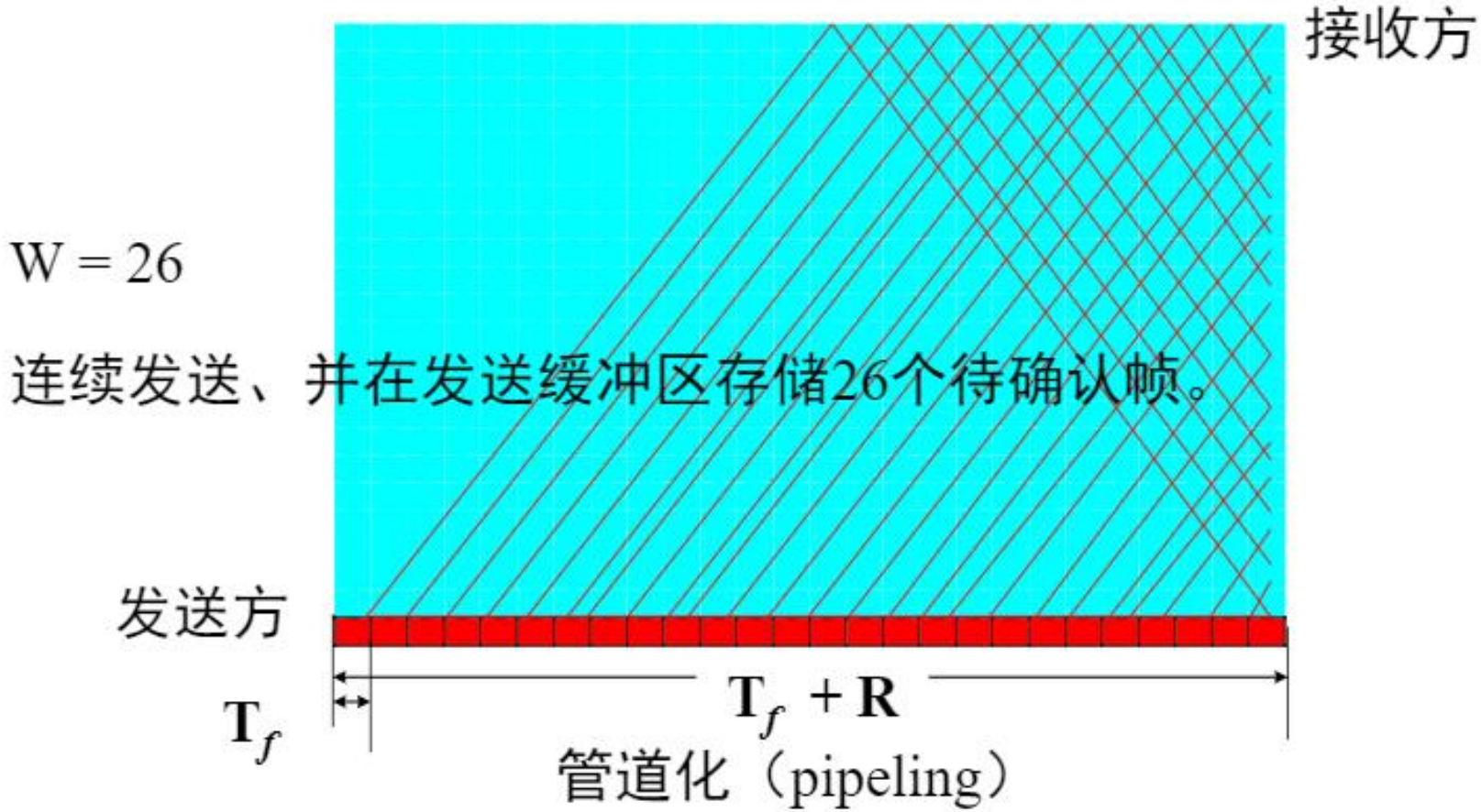
$$U = \frac{1000}{1000 + 25000} \approx 4\%$$

信道利用率不足4%



令： T_f = L/B

使信道利用率达到100%



提高信道利用率的方法

- 增加滑动窗口最大长度W

  信道利用率 $= W*T_f/(T_f+R)$

  $= W*k/(k+bR)$

- 理想情况下，使例题信道利用率达到100%，则滑动窗口最大长度为：

$$W = (T_f+R)/T_f = 520/20 = 26$$

注意这里符号和前面公式用的不一样，对应关系为

T_f := L/B

k := L

b := B

怎样找到一个合适的w值？

- 信道上的容量：一帧从发送方传输到接收期间可容纳的帧数量
- 带宽-延迟积：BD
- 窗口值：W=2*BD/L+1
- 上述的例子：
  - BD=50kbps*0.250=12.5kb
  - W=2*12.5kb+1=26kb=26帧

实际上：

$w \leq 2*BD+1$

注意：W=2*BD/L+1，其中，L=1000bits
这里的BD采用单向延迟（OWD）
进行计算，实际中经常用双向延迟RTT进行计算，此时不需要x2

课练

例（2014考研题）

主机甲和主机乙之间使用后退N帧协议（GBN）传输数据，甲的发送窗口尺寸为1000，数据帧长为1000字节，信道为100Mbps，乙每收到一个数据帧立即利用一个短帧（忽略其传输延迟）进行确认。若甲乙之间的单向传播延迟是50ms，则甲可以达到的最大平均传输速率约为：

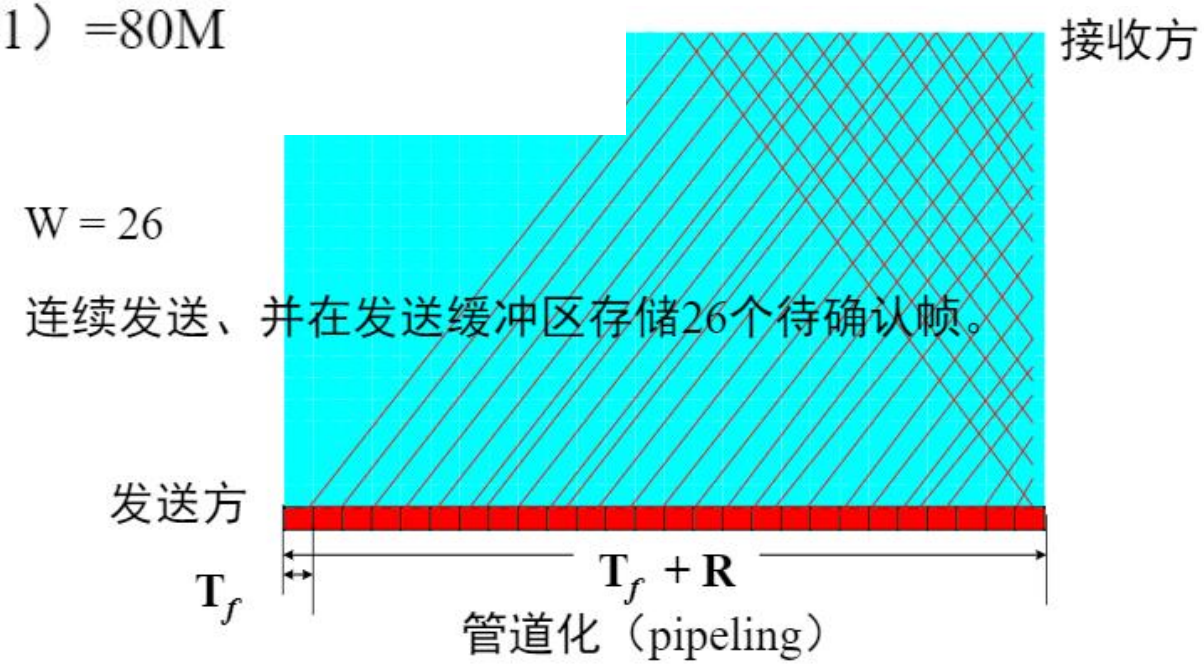
A. 10Mnps　　B. 20Mbps

C. 80Mbps　　D. 100Mbps

解

设可达到的最大传输率为 x，于是

- 1000f*1000Bpf*8=xbps*2*50ms/1000ms
- 8000000=8000+2*0.05x
- x=80Mbps
- 或者：w=1251，现在只有 1000，于是
- X=100M*（1000/1251）=80M



小结

- 批发数据时，其中一帧出错，可以从出错帧开始重传，也可以重传错误帧。
- 选择性重传需要接收方付出更多的缓存代价。
  - 适合出错率高的传输环境
- 选择性重传的窗口数
  - 发送窗口：$0 \le W \le (MAX_SEQ+1)/2$
  - 接收窗口：$W \le (MAX_SEQ+1)/2$
  - 一般地：接收窗口小于发送窗口

差错控制策略比较

- 回退n帧
  - 发送方需要较大的缓冲区，以便重传
  - 重传帧数多，适于信道出错率较少的情况
- 选择重传
  - 接收方需要较大的缓冲区，以便按正确顺序将分组提交网络层
  - 重传帧数少，适于信道质量不好的情况

3个协议的窗口大小

- One-Bit sliding window（协议4):
  - 0 <= size of Sending window<=1
  - size of receiving window=1
- Go-back-N (协议5)：
  - 0 <=size of Sending window<=MAX_SEQ
  - size of receiving window=1
- Selective Repeat (协议6)：
  - 0 <= size of Sending window<= (MAX_SEQ+1)/2
  - size of receiving window= (MAX_SEQ+1)/2

本章内容

3.1 数据链路层的设计问题

3.2 差错检测和纠正

3.3 基本的数据链路层协议

3.4 滑动窗口协议

3.5 数据链路协议实例

1. HDLC【自学】
2. 点到点链路层协议PPP【自学】
3. PPPoE 【自学】

高级数据链路控制规程HDLC （扩展阅读）

西南交通大学 Southwest Jiaotong University
计算机网络教案社区

一、系列协议

1. 面向字符的链路控制规程(连路上传送的数据信息和控制信息必须使规定的字符集，如ASCII)，典型的有：

- APANET的IMP–IMP协议；
- IBM的BSC规程(Binary Synchronous Communication)；

缺点：1) 采用停等协议，线路利用率低；

2) 所有通信设备必须使用相同的字符代码；

3) 只对数据部分进行差错控制；

4) 不宜扩展，增加功能需增加新的控制字符。

2. 面向比特的链路控制规程

- SDLC协议：IBM SNA体系结构的数据链路层协议。
- HDLC(High-level Data Link Control)协议：ISO协议。SDLC协议稍加修改，
- LAP (Link Access Procedure)：CCITT将HDLC修改后称链路接入规程LAP。
- LAPB协议：HDLC的新版本把LAP修改为LAPB，“B”表示平衡型(Balanced)。
  - 大同小异；
  - 基于HDLC产生其他多种类似协议，如PPP、LLC等。

本章内容

3.1 数据链路层的设计问题

3.2 差错检测和纠正

3.3 基本的数据链路层协议

3.4 滑动窗口协议

3.5 数据链路协议实例

1. HDLC
2. 点到点链路层协议PPP
3. PPPoE

PPP协议简介

- PPP (Point-to-Point Protocol)协议由IETF制定，1994年成为正式标准（RFC1661）
- PPP协议是目前使用最多的数据链路层协议之一
- 能够在不同的链路上运行
- 能够承载不同的网络层分组
- 特点：简单、灵活

PPP协议实现的功能

- 利用帧定界符封装成帧
- 填充技术实现透明数据传输：字节填充、零比特填充
- 帧的差错检测
- 实时监测链路工作状态
- 设置链路最大传输单元（MTU）
- 网络层地址协商机制
- 数据压缩协商机制

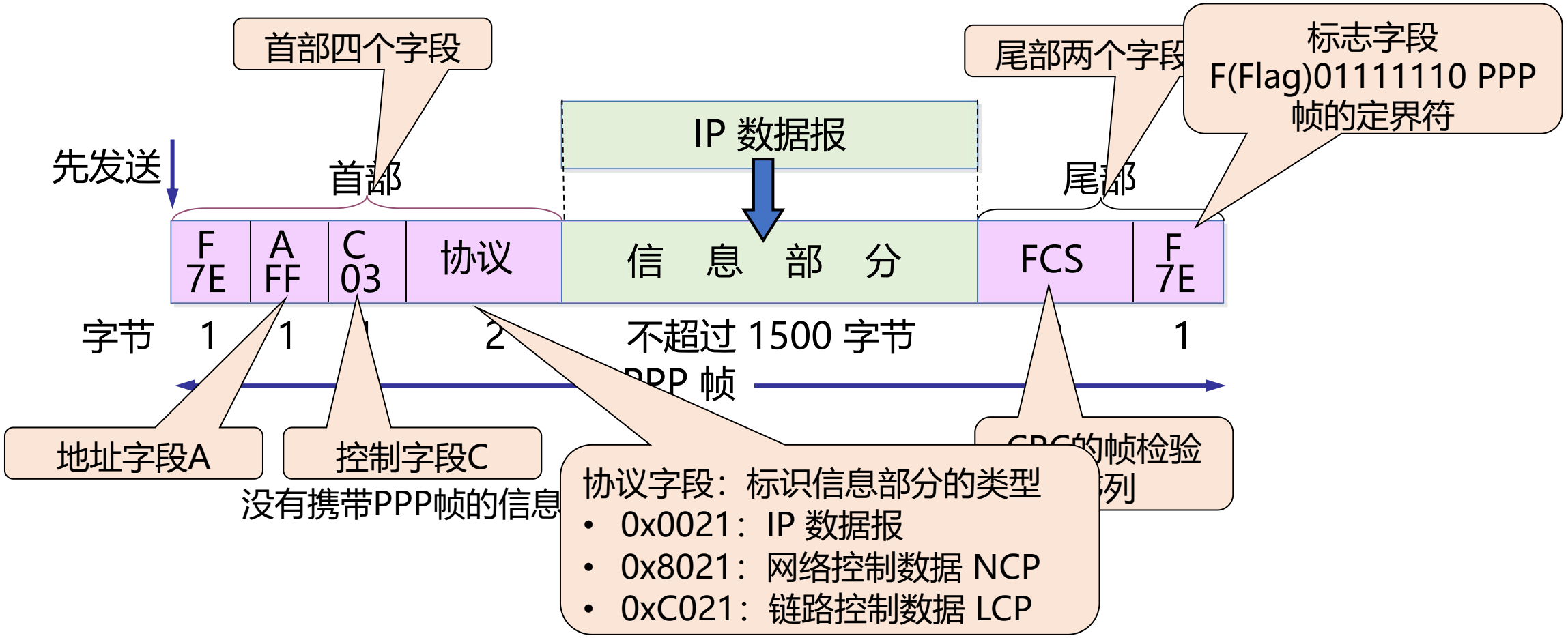
PPP协议未实现的功能

- 帧数据的纠错功能
  - 数据链路层的PPP协议只进行检错，PPP协议是不可靠传输协议
- 流量控制功能
  - PPP协议未实现点到点的流量控制
- 可靠传输功能
  - PPP为不可靠协议，不使用帧的序号。不可靠网络中可能使用有序号的工作方式。
- 多点连接功能
  - PPP协议不支持多点线路，只支持点对点的链路通信。
- 单工和半双工链路
  - PPP协议支持全双工链路

PPP协议的构成

- 封装 (Encapsulation)
  - 提供在同一链路上支持不同的网络层协议
  - PPP既支持异步链路（无奇偶检验的8比特数据），也支持面向比特的同步链路
  - IP数据包在PPP帧中是其信息部分，其长度受到MTU的限制
- 链路控制协议 LCP (Link Control Protocol)。
  - 用来建立、配置和测试数据链路的链路控制协议，通信双方可协商一些选项
- 网络控制协议 NCP (Network Control Protocol)。
  - 其中每个协议支持一种不同的网络层协议，如IP、OSI的网络层、DECnet、AppleTalk等

PPP协议的帧格式


首部四个字段
尾部两个字段
标志字段
F(Flag)01111110 PPP
帧的定界符
IP 数据报
先发送
首部
尾部
F
7E
A
FF
C
03
协议
信 息 部 分
FCS
F
7E
字节 1 1 1 2 不超过 1500 字节 2 1
PPP 帧
地址字段A
控制字段C
没有携带PPP帧的信息
协议字段：标识信息部分的类型
• 0x0021：IP 数据报
• 0x8021：网络控制数据 NCP
• 0xC021：链路控制数据 LCP

PPP协议的帧格式

- PPP协议中的透明传输
  - 当 PPP 用在异步传输时，使用一种特殊的字符填充法
  - 当 PPP 用在同步传输链路时，协议规定采用硬件来完成比特填充（与 HDLC 的做法类似）

PPP协议的帧格式

➢ 字节填充

- 避免在信息字段中出现和标志字段一样的比特组合（0X7E）
- 当PPP使用异步传输时，定义转义字符0X7D，并使用字节填充
  - 将信息字段中出现的0x7E 字节转变成为2字节序列(0x7D, 0x5E)
  - 若信息字段中出现一个0x7D 字节, 则将其转变成为2字节序列(0x7D, 0x5D)
  - 若信息字段中出现 ASCII 码的控制字符（即数值小于 0x20 的字符），则在该字符前面要加入一个 0x7D 字节，同时将该字符的编码加以改变
- 在发送端进行了字节填充，链路上传递的信息字节数超过原来的信息字节数。接收端进行字节填充相反的工作，恢复为原来的信息

PPP协议的帧格式

➢ 字节填充

- 0x7e 字节填充后编码为 0x7d, 0x5e.　(Flag Sequence)帧序列
- 0x7d 字节填充后编码为 0x7d, 0x5d.　(Control Escape)控制转义符
- 0x03 字节填充后编码为 0x7d, 0x23.　(ETX)文本结束符
- 0x11 字节填充后编码为 0x7d, 0x31.　(XON)重新传输
- 0x13 字节填充后编码为 0x7d, 0x33.　(XOFF)传输暂停
- 0x91 字节填充后编码为 0x7d, 0xb1.　(XON with parity set)带奇偶校验集合的XON
- 0x93 字节填充后编码为 0x7d, 0xb3.　(XOFF with parity set)带奇偶校验集合的XOFF

PPP协议的帧格式

- 零比特填充
  - PPP协议用在SONET/SDH链路时，使用同步传输（一连串的比特连续传送），不是异步传输（逐个字符地传送）
    - 采用比特填充方法来实现透明传输。
    - 信息字段中出现了和标志字段 F 完全一样的 8 比特组合（0x7E）；
    - 发送端在 5 个连 1 之后填入 0 比特再发送出去；
    - 在接收端把 5 个连 1之后的 0 比特删除

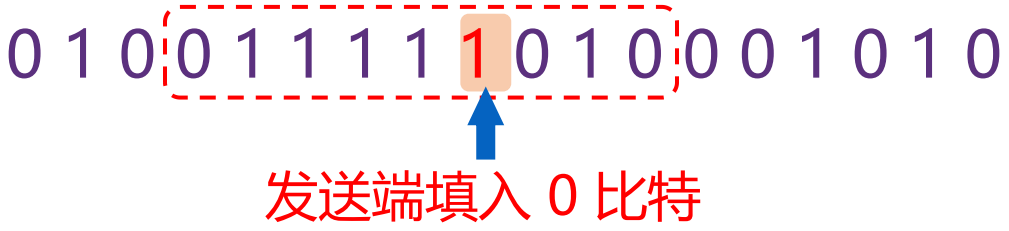
PPP协议的帧格式

➢ 零比特填充

1. 信息字段中出现了和标志字段 F 完全一样的 8 比特组合



2. 发送端在 5 个连 1 之后填入 0 比特再发送出去



3. 在接收端把 5 个连 1之后的 0 比特删除

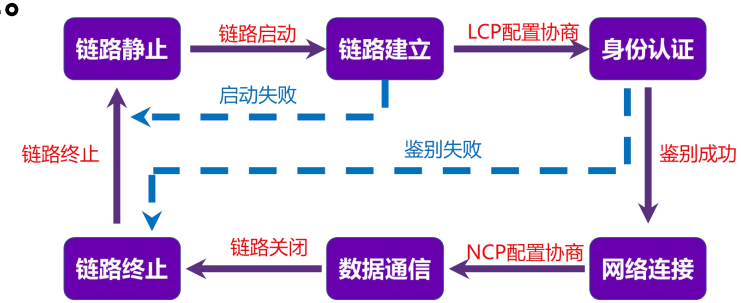


PPP协议的工作状态及转换

- 在点对点链路上建立通信，通信双方通过LCP报文进行配置和测试链路
- 对等实体进行身份认证（可选）
- PPP协议必须发送NCP报文来选择和配置一个或多个网络层协议
- 双方网络层协议可以通过PPP链路发送数据报，并持续通信
- LCP或NCP数据报显式关闭链路，或者异常事件导致关闭链路

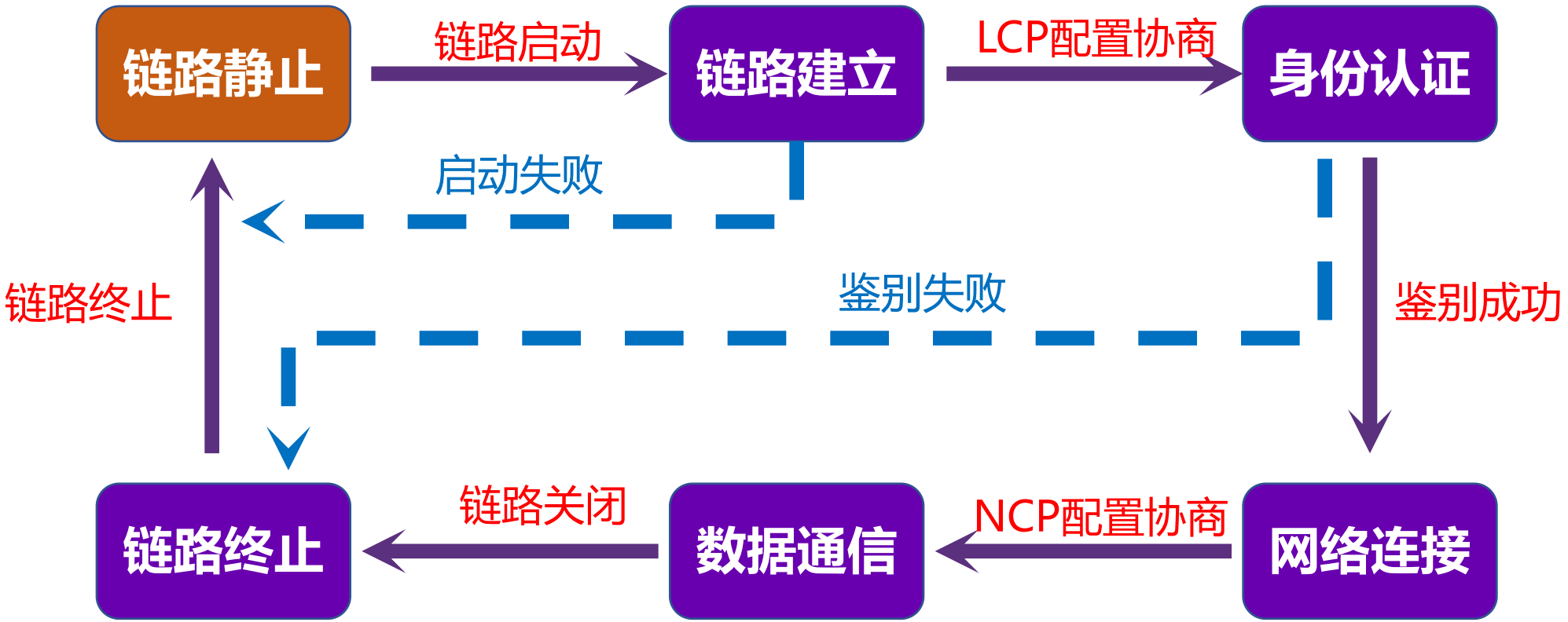
PPP协议的工作状态及转换

- 状态转换图(State Transform Diagram)
  - 状态转换图通过描述系统的状态和引起系统状态转换的事件，来表示系统的行为，指出作为特定事件的结果将执行哪些动作。描述系统对内部事件和外部事件如何响应，如何动作。
  - 状态转换图中以节点表示状态，有向边来表示内外部事件和响应。
  - 本节将采用状态转换图来表示PPP协议工作的行为模型



- 状态
  - PPP协议的状态转换图中共设置了六个状态：
  - 链路静止、链路建立、身份认证、网络连接、数据通信、链路终止
- 状态的变迁
  - PPP协议中通过一些外部事件来触发状态的变迁，如：
  - 链路启动、LCP配置协商、鉴别成功、NCP配置协商、链路关闭、链路终止等

PPP协议的工作状态及转换

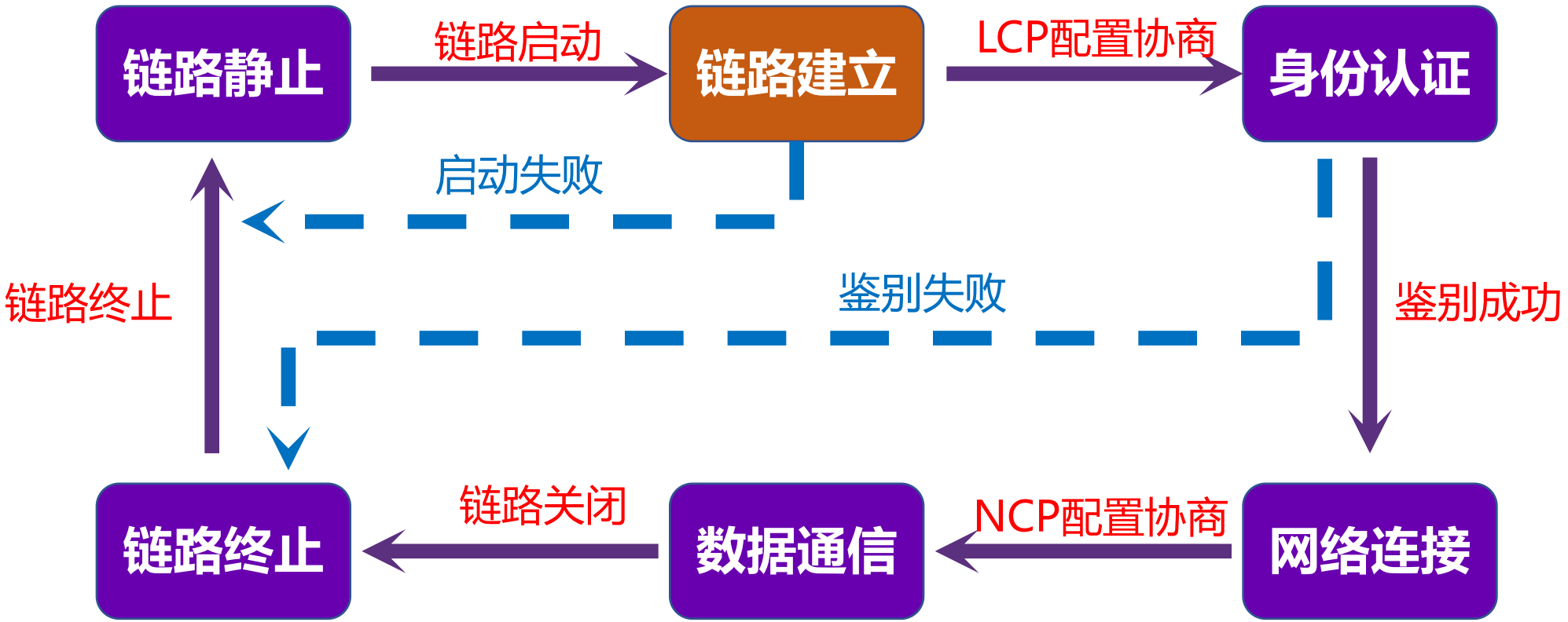


PPP协议的工作状态及转换

➢ 链路静止状态

- PPP链路的起始和终止状态，通信双方尚无链路
- 当外部事件表明物理层可以使用时，PPP将进入链路建立阶段。
  - 如载波检测、网络管理员配置
  - PC机通过调制解调器呼叫路由器时，路由器能检测到调制解调器发出的载波信号，并作出应答。

PPP协议的工作状态及转换



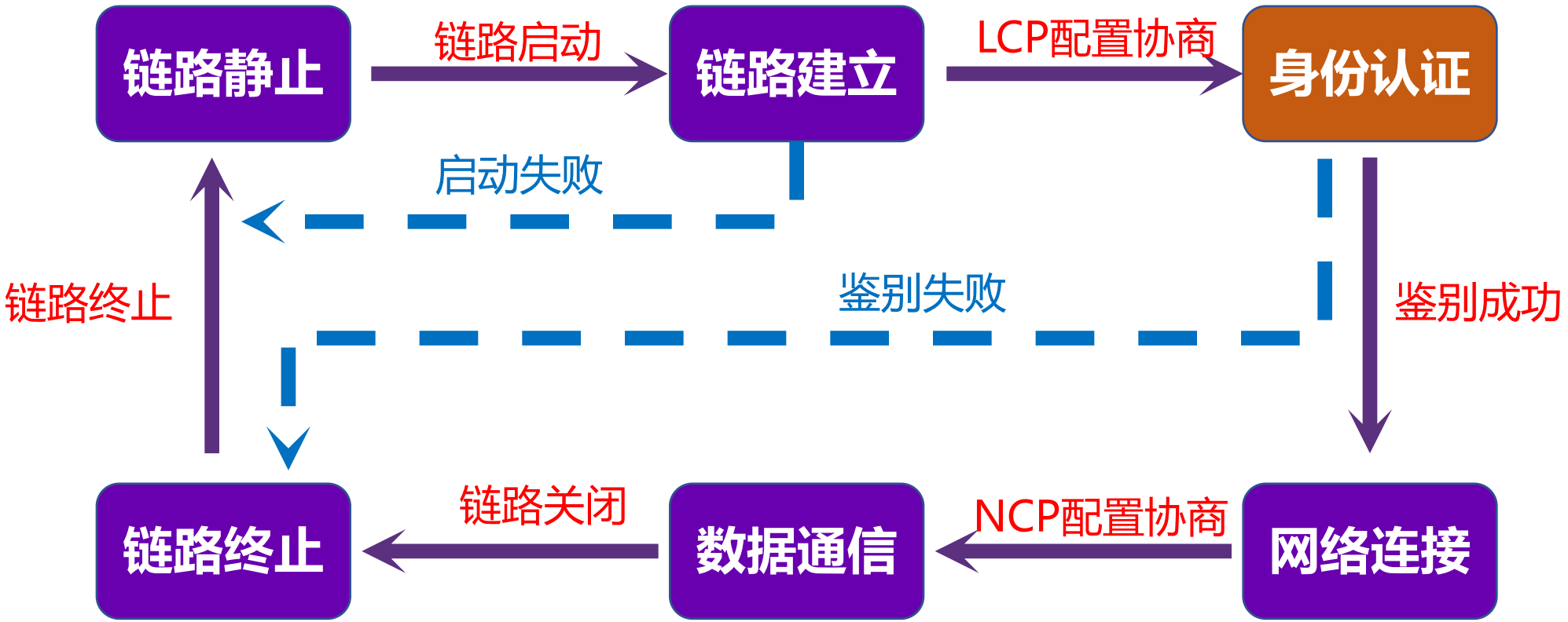
PPP协议的工作状态及转换

- 链路建立状态
  - LCP通过交换配置包来建立连接
  - LCP配置选项包括：链路上的最大帧长、所使用的鉴别协议的规约、不使用PPP帧中的地址和控制字段
  - LCP开始协商一些配置选项，发送LCP的配置请求帧（Configure-Request），协议字段为LCP对应的代码，信息字段包含特定的配置请求
  - 链路的另一端可以发送几种响应的一种
    - 配置确认帧（Configure-Ack）：所有选项都接受
    - 配置否认帧（Configure-Nak）：所有选项都理解但是不能接受
    - 配置拒绝帧（Configure-Reject）：选项有的无法识别或不能接受，需要协商

PPP协议的工作状态及转换

- 链路建立状态
  - LCP帧共有11种（以上为4种配置型帧）。
    - 关闭线路：Terminate-request请求关闭线路帧，Terminate-ack接受关闭线路；
    - 应答方无法理解：Code-reject收到了未知的请求，Protocol-reject被请求的协议未知； 可能是传输错误或者LCP协议版本不同
    - 用于测试线路质量：Echo-request回送请求帧，Echo-reply回送应答帧；
    - 用于测试：Discard-request只需丢弃该帧

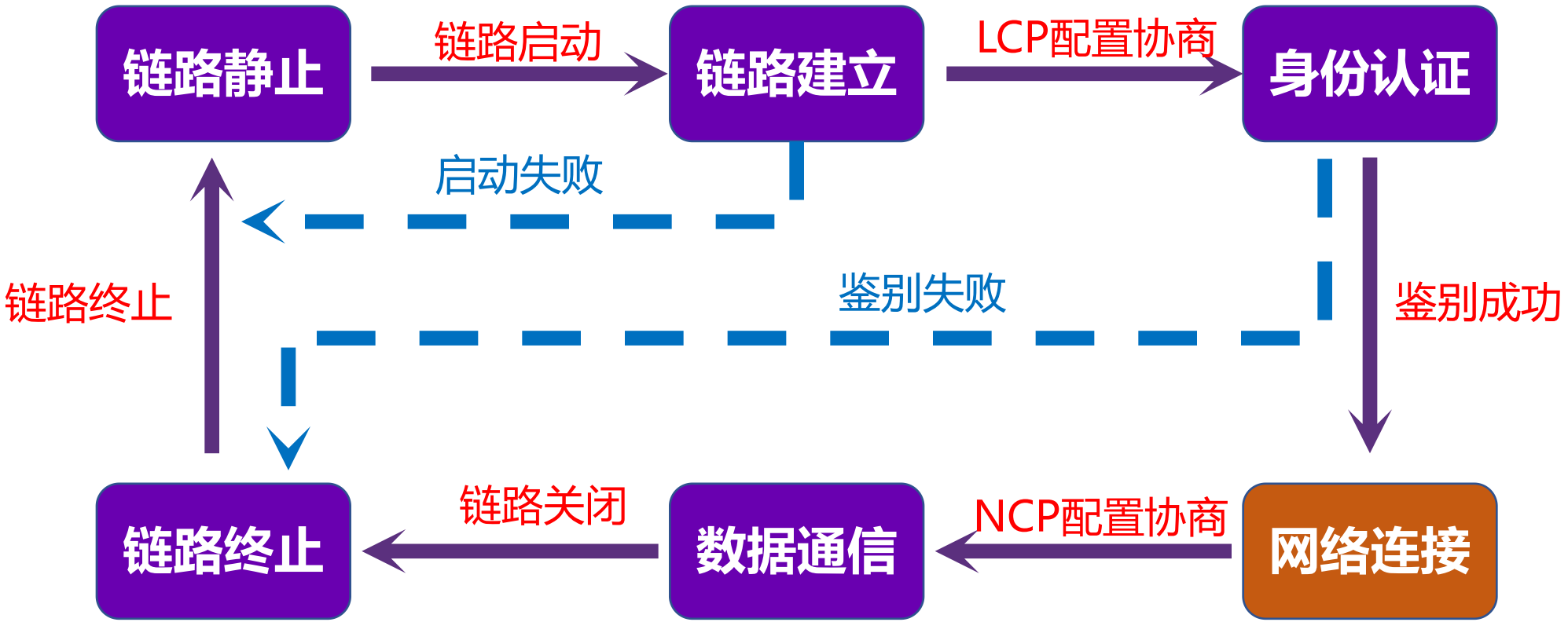
PPP协议的工作状态及转换



PPP协议的工作状态及转换

- 身份认证状态
  - 在允许交换网络层协议包之前，对等点可选身份验证
  - 默认情况下，不需要身份验证
  - 如果选择身份验证协议，需要在链接建立阶段请求使用该身份验证协议
  - 认证将在链接建立后进行，链路质量确认可能会同时发生
  - 在身份验证完成之前，不能从身份验证阶段推进到网络层协议阶段
  - 如果身份验证失败，则身份验证器应该继续到链接终止阶段
  - 此阶段只允许链路控制协议、认证协议和链路质量监控包
  - 此阶段接收到的所有其他数据包必须丢弃

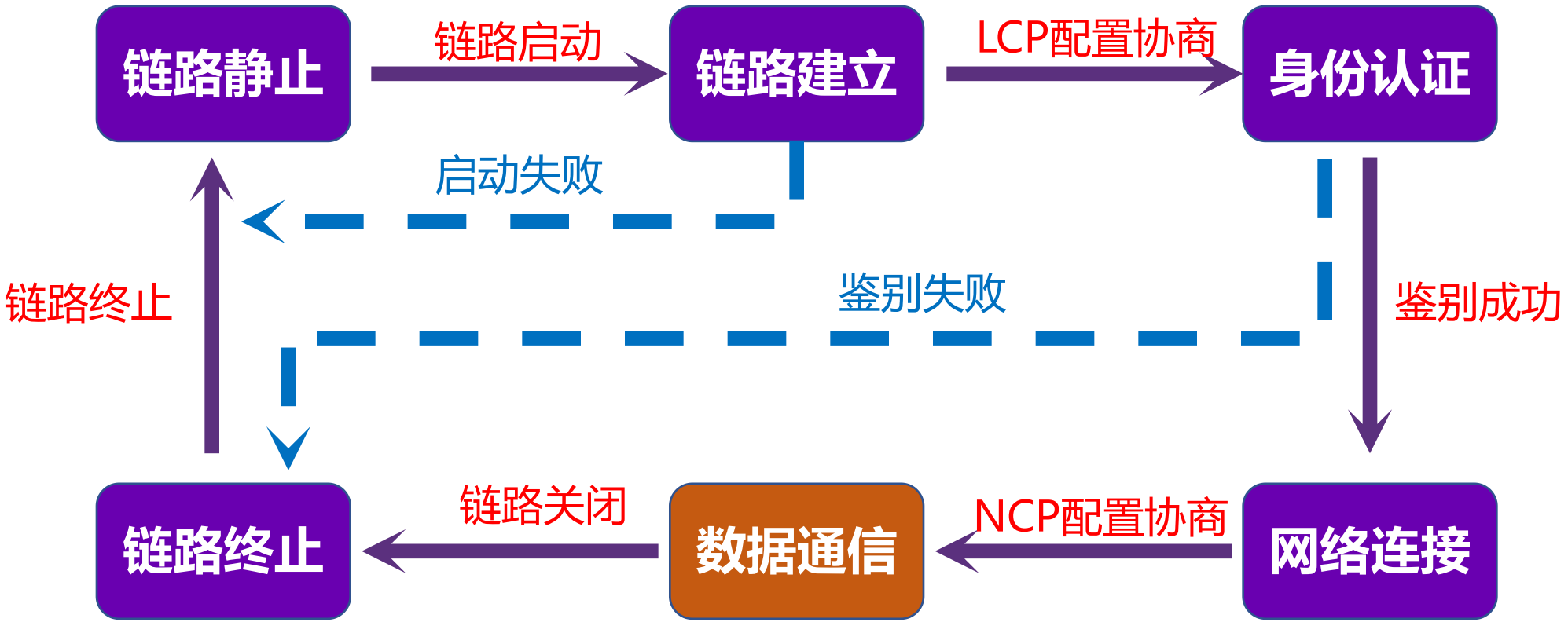
PPP协议的工作状态及转换



PPP协议的工作状态及转换

- 网络连接状态
  - 完成了前面的阶段，每个网络层协议(如IP、IPX或AppleTalk)必须由适当的网络控制协议(NCP)单独配置
  - 每个NCP可以在任何时候打开和关闭
  - NCP到达打开状态后，PPP将携带相应的网络层协议包
  - 如果相应的NCP不处于打开状态时，接收到的任何受支持的网络层协议包都必须丢弃
  - 在此阶段，链路流量由LCP、NCP和网络层协议包的任何可能组合组成

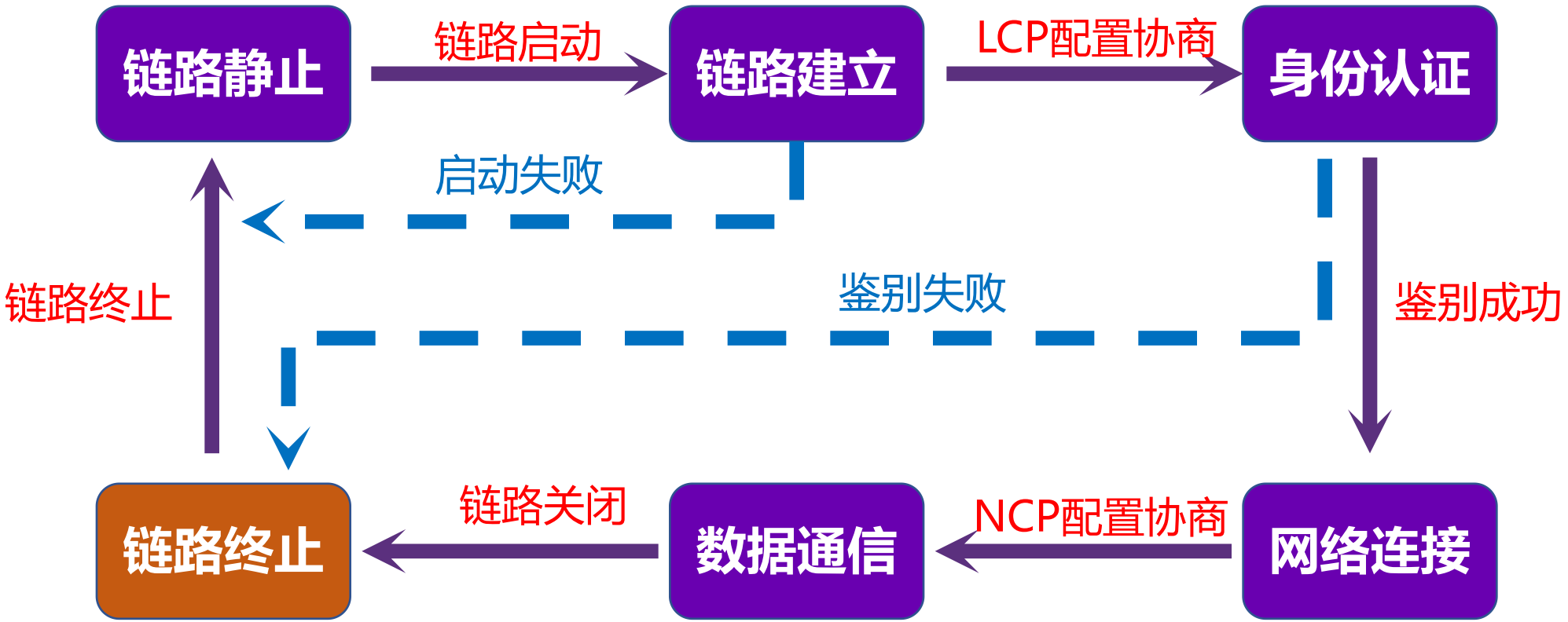
PPP协议的工作状态及转换



PPP协议的工作状态及转换

- 数据通信状态（网络打开状态）
  - 链路的两个PPP端点可以彼此向对方发送分组
  - 或者发送检查链路状态的Echo-*帧

PPP协议的工作状态及转换



PPP协议的工作状态及转换

- PPP可以在任何时候终止链接。
  - 运营商丢失、身份验证失败、链接质量失败、空闲期计时器过期或链接的管理关闭造成的
- LCP用于通过交换终止包来关闭链接。
  - 当连接关闭时，PPP通知网络层协议
  - 在交换终止包之后，实现应该向物理层发出断开的信号，以强制终止链接，特别是在身份验证失败的情况下
- Terminate-Request的发送方在收到Terminate-Ack或重启计数器过期后断开连接。
- Terminate-Request的接收方等待对等方断开连接，并且在发送Terminate-Ack之后，在至少一次重新启动时间过去之前，绝不能断开连接。
- PPP随后进入链路静止阶段。
- 在此阶段接收到的任何非LCP数据包都必须丢弃。

本章内容

3.1 数据链路层的设计问题

3.2 差错检测和纠正

3.3 基本的数据链路层协议

3.4 滑动窗口协议

3.5 数据链路协议实例

1. HDLC
2. 点到点链路层协议PPP
3. PPPoE

PPPoE概述

- Ethernet优点
  - 原理简单，应用非常广，设备成本低
- Ethernet缺点
  - 安全性较低、不宜管理：使用广播信道，造成了安全性较低，无认证功能
- PPP优点
  - 原理简单
  - 安全性高：点对点信道，提供认证机制
  - 提供良好的访问控制和计费功能

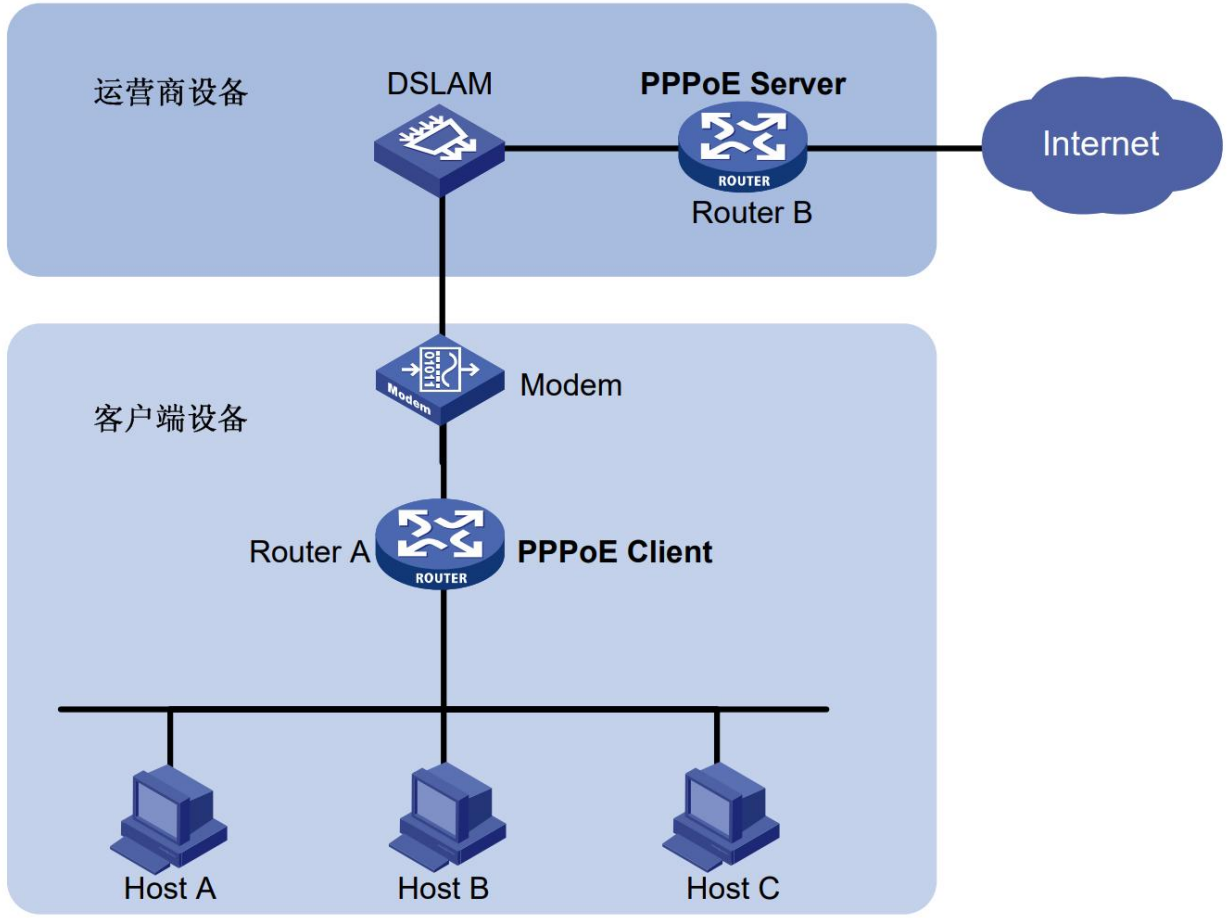
PPPoE概述

- PPPoE（Point-to-Point Protocol over Ethernet）
  - 提供在以太网链路上的PPP连接
  - 实现了传统以太网不能提供的身份验证、加密，以及压缩等功能
  - 实现基于用户的访问控制、计费、业务类型分类等，运营商广泛支持
  - PPPoE使用Client/Server模型，服务器通常是接入服务器

PPPoE组网方式

➢ 组网方式1

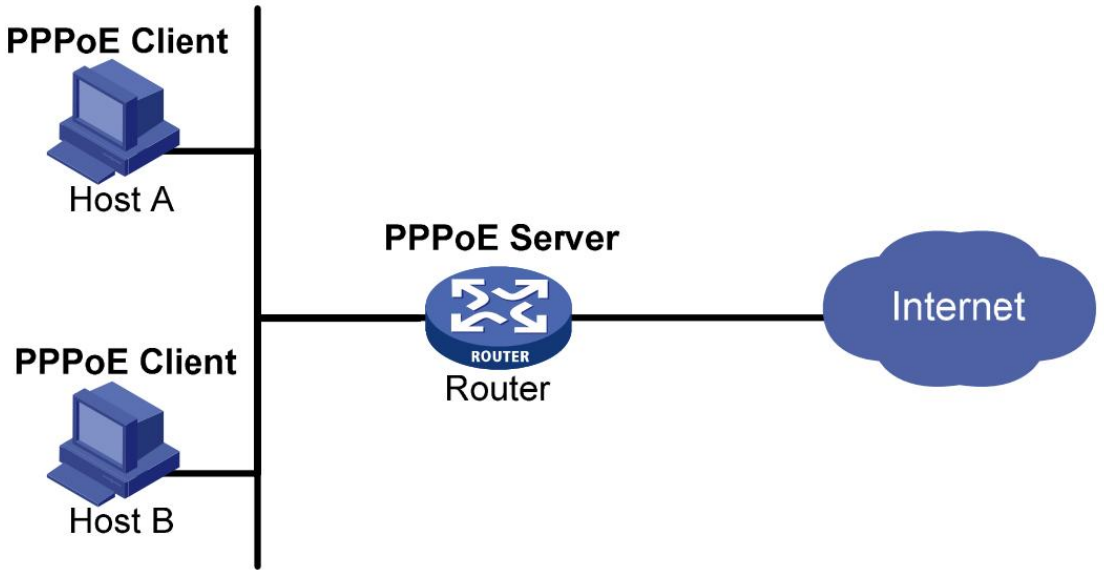
- 设备之间建立 PPP 会话，所有主机通过同一个 PPP 会话传送数据，主机上不用安装 PPPoE 客户端拨号软件，一般是一个机构共用一个账号
- PPPoE Client 位于机构内
- PPPoE Server 是运营商的设备



PPPoE组网方式

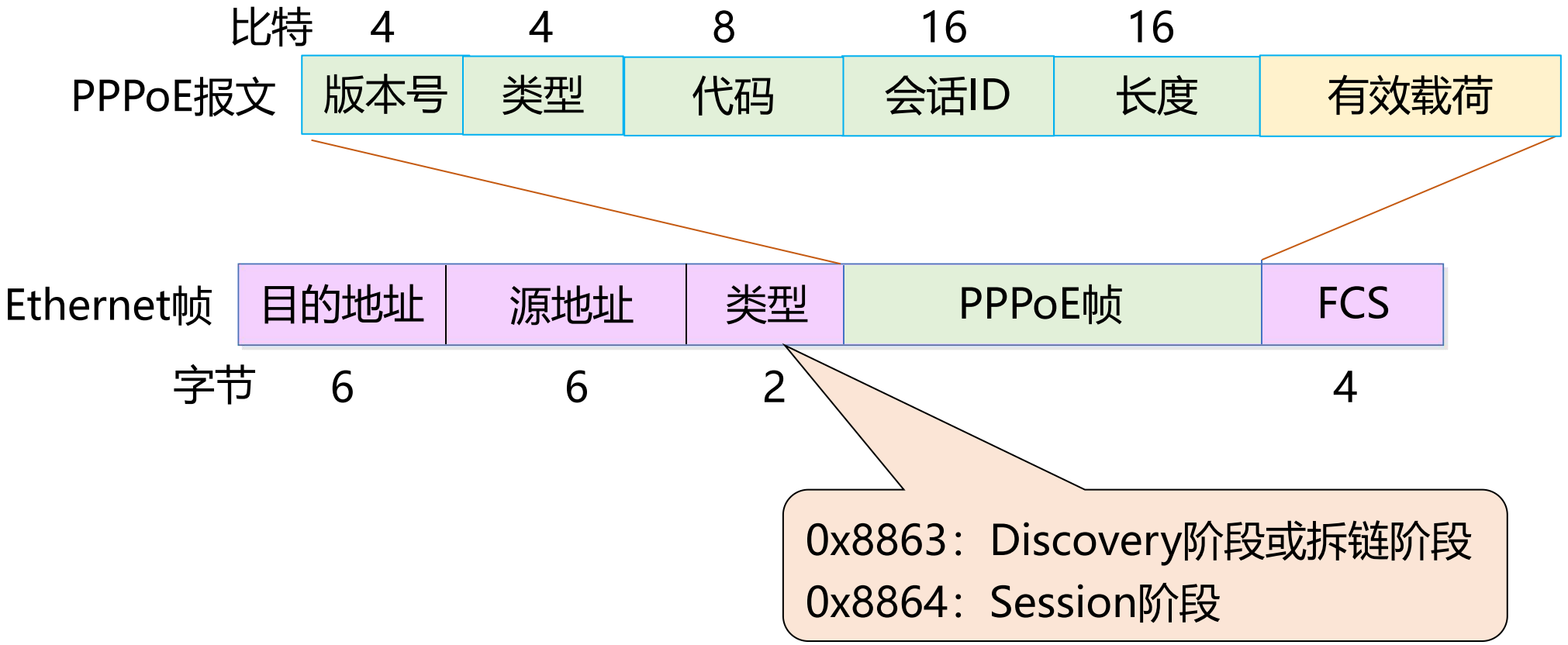
➢ 组网方式2

- PPP 会话建立在 主机和运营商的路由器之间，为每一个主机建立一个 PPP 会话，每个主机都是 PPPoE Client，每个主机有一个帐号，方便运营商对用户进行计费和控制
- 主机上需要安装PPPoE 客户端软件



PPPoE报文格式

- PPPoE报文格式及Ethernet帧封装



PPPoE报文格式

- 版本号：PPPoE 版本号，值为 0x1
- 类型：PPPoE 类型，值为 0x1
- 代码：PPPoE 报文类型
- 会话ID：
  - 分配的PPPoE会话ID
  - 与以太源地址和目的地址一起定义了一个会话
- 长度：定义有效载荷长度

代码值	PPPoE 报文类型
0x00	会话数据
0x09	PADI 报文
0x07	PADO 或 PADT 报文
0x19	PADR 报文
0x65	PADS 报文

PPPoE工作过程

- PPPoE可分为三个阶段：
  - Discovery阶段
    - 获取对方以太网地址，确定PPPoE会话ID
  - Session阶段
    - PPP协商阶段
    - PPP报数数据传输
  - Terminate阶段
    - 会话建立以后的任意时刻，发送报文结束会话

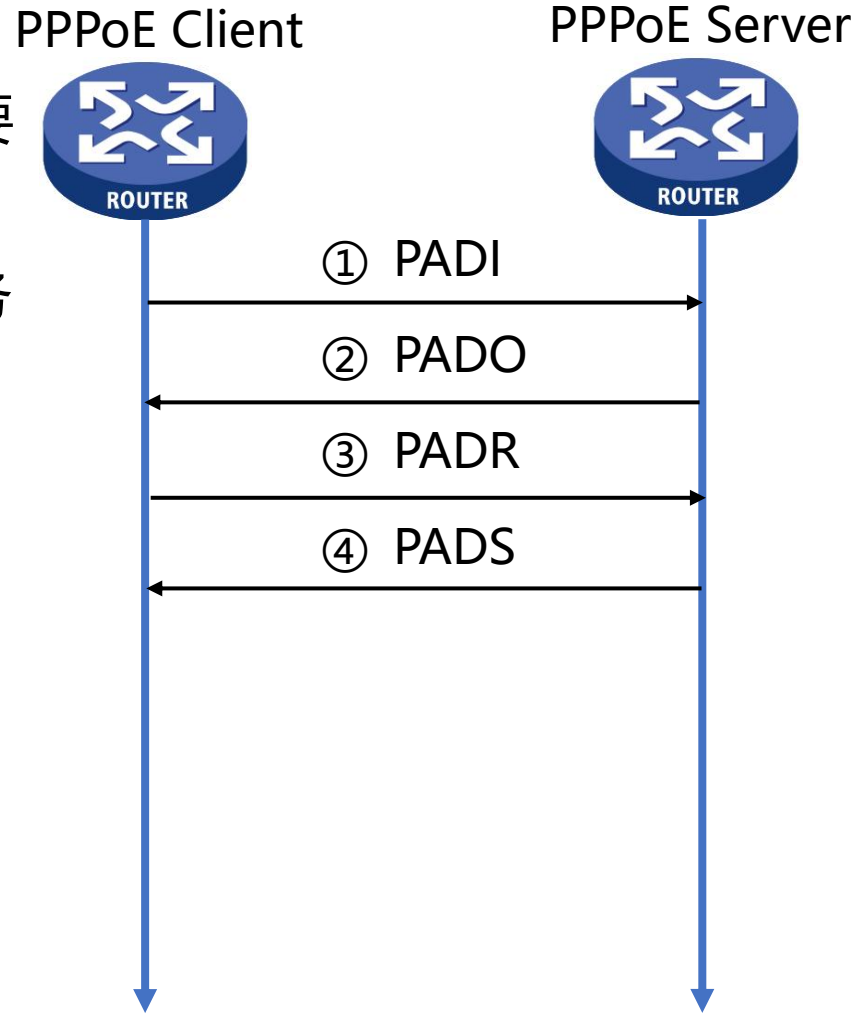
PPPoE工作过程

- Discovery阶段和Terminate阶段使用的报文类型

类型	描述
PADI（PPPoE Active Discovery Initiation）	PPPoE发现初始报文
PADO（PPPoE Active Discovery Offer）	PPPoE发现提供报文
PADR（PPPoE Active Discovery Request）	PPPoE发现请求报文
PADS（PPPoE Active Discovery Session-confirmation）	PPPoE发现会话确定报文
PADT（PPPoE Active Discovery Terminate）	PPPoE发现终止报文

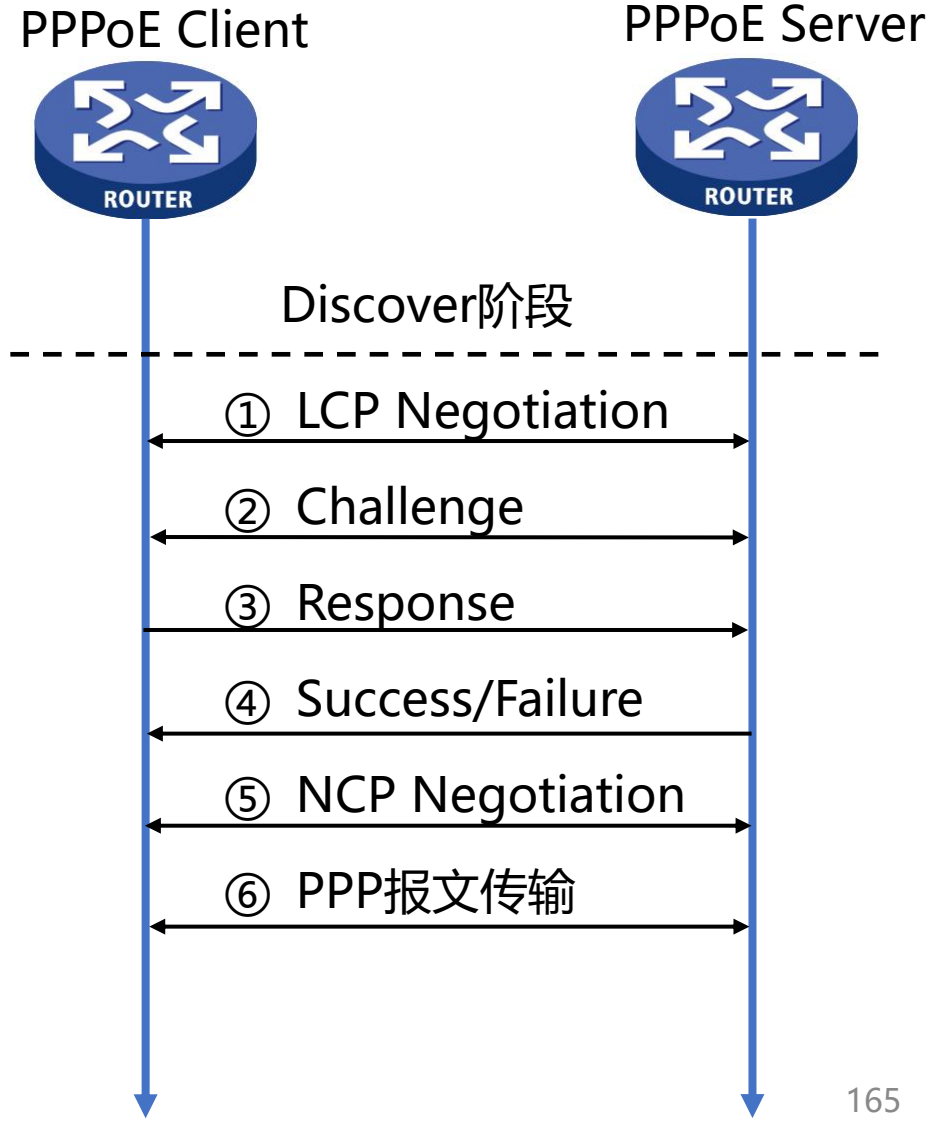
PPPoE工作过程

- Discovery阶段：
  - PPPoE Client广播PADI报文，报文中包含PPPoE Client 想要得到的服务类型信息
  - 所有的PPPoE Server收到PADI报文之后，将其中请求的服务与自己能够提供的服务进行比较，如果可以提供，则单播回复PADO报文
  - 根据网络的拓扑结构，PPPoE Client可能收到多个PPPoE Server发送的PADO报文，PPPoE Client选择最先收到PADO报文对应的 PPPoE Server，并单播PADR报文
  - PPPoE Server 产生唯一的Session ID，标识与 PPPoE Client 的会话，发送PADS报文把会话ID发送给PPPoE Client，进入Session阶段



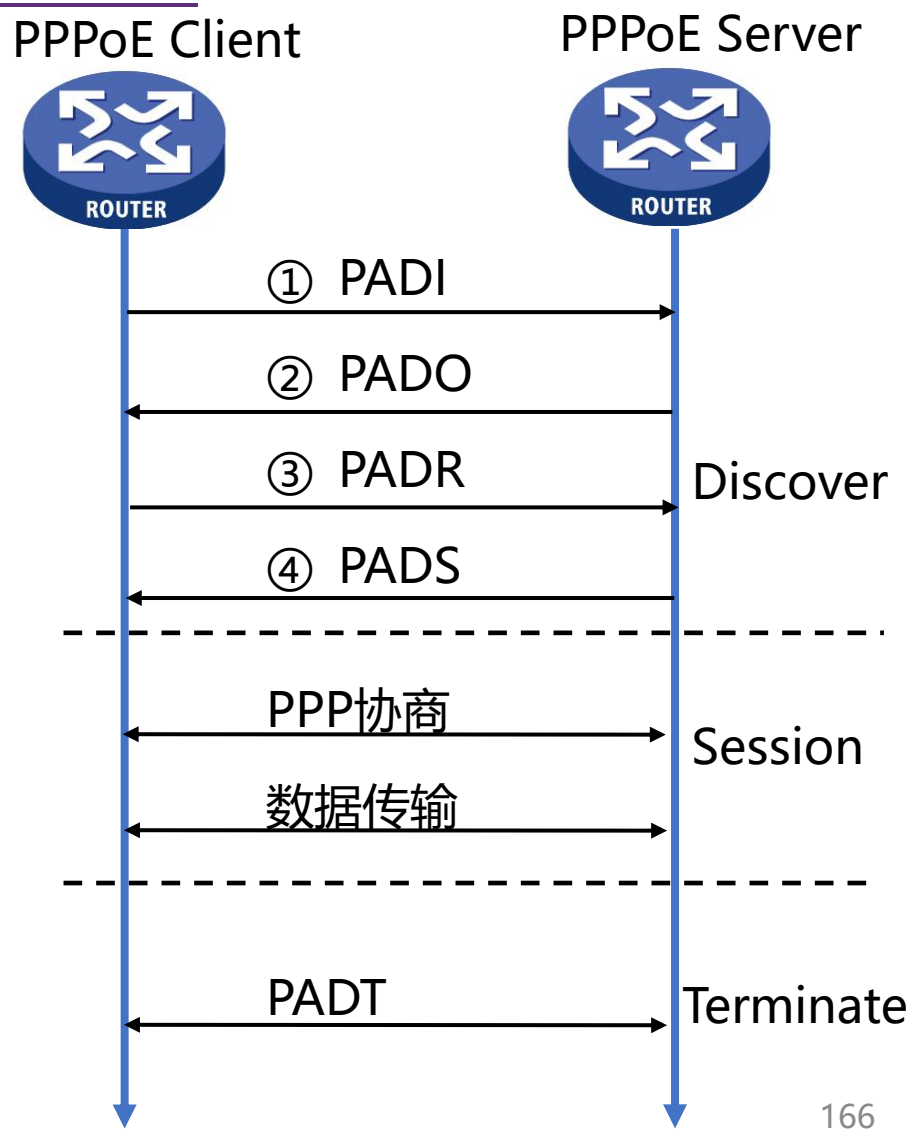
PPPoE工作过程

- Session阶段：
  - PPP协商阶段
    - 与普通的PPP协商方式一致，分为LCP、认证、NCP三个阶段
    - LCP 阶段主要完成建立、配置和检测链路连接。LCP 协商成功后，开始进行认证工作，认证协议类型由 LCP 协商结果决定
    - 认证成功后，进入NCP阶段。NCP是一个协议族，用于配置不同的网络层协议，常用的是IP控制协议(IPCP)，负责配置用户的IP和DNS等工作
  - PPP报文传输阶段



PPPoE工作过程

- Terminate阶段，终止方式有两种：
  - PPP协议自身（比如PPP终结报文）来结束PPPoE会话
  - 使用PADT报文终结会话
    - 进入PPPoE Session阶段后，PPPoE Client和PPPoE Server都可以通过发PADT报文的方式来结束PPPoE连接
    - PADT数据包可以在会话建立以后的任意时刻单播发送。



本章总结

- 通过本章学习，重点掌握如下知识点：
  - 理解数据链路层在五层网络体系结构中的位置及解决的问题，了解数据链路层提供的服务类型
  - 掌握差错检测和纠正的基本原理和典型的编码方法
  - 掌握无错信道和有错信道上停等协议的设计和实现方法
  - 理解停等协议的性能问题及滑动窗口协议的基本思想
  - 掌握回退N和选择重传两种典型滑动窗口协议的工作机制
  - 掌握点到点链路层协议PPP与PPPoE的工作机制

习 题 (作业交1、2、3、6-9)

1、某一个数据通信系统采用CRC校验方式，生成多项式G(x)的二进制比特序列为1101，目的地接受到的二进制比特序列为110111001(含校验码)，请问传输过程中是否出现差错？为什么？

2、一个帧由100个8比特字符组成，使用下列方案在一条数据链路上传输，需要多少附加的比特？

(a)**字符同步**：每个字符使用一个起始位和两个停止位，每个报文使用一个帧起始字符和一个帧结束字符；

(b) **帧同步**：每个帧使用两个同步字符(一个帧起始字符和一个帧结束字符)。

3、在一个1Mb/s的卫星信道上发送1000比特长的帧。确认总是捎带在数据帧中。帧头很短，使用3位的序列号。对以下协议而言，可以取得的最大信道利用率是多少？ (a)停等协议；(b)回退N滑动窗口协议；(c)选择重传滑动窗口协议。(卫星信道端到端的传输延迟是270ms)

4、要发送的数据为1101011011，采用CRC的生成多项式是$G(X)=X^4+X+1$，试求CRC编码后的传输码，数据在传输过程中最后一个1变成了0，试问接收端是如何发现的？

5、信道速率为4kb/s，采用停等协议，传播时延t_p=20ms。确认帧长度和处理时间均可忽略。求使信道利用率达到至少50%的帧长度。

6、分析停止等待协议中的确认帧是否需要序号，说明理由。

7、举例说明：当用n个比特进行编号时，若接收窗口大小为1，则只有在发送窗口的大小 $W_T \leq 2^n-1$ 时，连续ARQ协议才能正确运行。

8、在选择重传ARQ协议中，设编号用3比特，发送窗口W_T=5，接收窗口W_R=4。举出使协议不能正确工作的一种情况。

9、在连续ARQ协议中，设编号用3比特，发送窗口W_T=8。试找出使协议不能正确工作的一种情况。